

Towards Robustness : Adversarial Attack on Bangla Language Models

A Thesis

Submitted in partial fulfillment of the requirements for the Degree of
Bachelor of Science in Computer Science and Engineering

Submitted by

Sheikh Fardeen Ishaque	190104011
SM Musfiqur Rahman	20200104002
Ziaul Hassan	20200104004
MD Rafsanul Islam Nelay	20200104079
Mohammad Rabbu	20200104109

Supervised by

Mr. Sajib Kumar Saha Joy



Department of Computer Science and Engineering
Ahsanullah University of Science and Technology

Dhaka, Bangladesh

December 2024

CANDIDATES' DECLARATION

We, hereby, declare that the Thesis presented in this report is the outcome of the investigation performed by us under the supervision of Mr. Sajib Kumar Saha Joy, Department of Computer Science and Engineering, Ahsanullah University of Science and Technology, Dhaka, Bangladesh. The work was spread over two final year courses, CSE4100: Project and Thesis I and CSE4250: Project and Thesis II, in accordance with the course curriculum of the Department for the Bachelor of Science in Computer Science and Engineering program.

It is also declared that neither this Thesis nor any part thereof has been submitted anywhere else for the award of any degree, diploma or other qualifications.

Sheikh Fardeen Ishaque

Sheikh Fardeen Ishaque
190104011

SM Musfiqur Rahman

SM Musfiqur Rahman
20200104002

Ziaul

Ziaul Hassan
20200104004

MD Rafsanul Islam Nelay

MD Rafsanul Islam Nelay
20200104079

Mohammad Rabbu

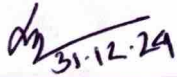
Mohammad Rabbu
20200104109

CERTIFICATION

This Thesis titled, **"Towards Robustness : Adversarial Attack on Bangla Language Models"**, submitted by the group as mentioned below has been accepted as satisfactory in partial fulfillment of the requirements for the degree B.Sc. in Computer Science and Engineering in December 2024.

Group Members:

Sheikh Fardeen Ishaque	190104011
SM Musfiqur Rahman	20200104002
Ziaul Hassan	20200104004
MD Rafsanul Islam Nelay	20200104079
Mohammad Rabbu	20200104109


31.12.24

Mr. Sajib Kumar Saha Joy
Lecturer Grade-I & Supervisor
Department of Computer Science and Engineering
Ahsanullah University of Science and Technology


AKHTER

Prof. Dr. Md. Shamim Akhter
Professor & Head
Department of Computer Science and Engineering
Ahsanullah University of Science and Technology

ACKNOWLEDGEMENT

We would like to express our deepest gratitude to our supervisor, Mr. Sajib Kumar Saha Joy, for his invaluable guidance, constant support, and encouragement throughout the course of this thesis. His expertise and meticulous attention to detail have greatly enriched our work and inspired us to strive for excellence.

We would also like to extend our appreciation to the Department of Computer Science and Engineering and the esteemed faculty members for their constructive feedback and valuable suggestions during the course of this research. A special note of thanks goes to our group members, whose collaboration, support, and encouragement provided us with the motivation to persevere.

Finally, we would like to express our heartfelt gratitude to everyone who contributed to this thesis in any capacity. Your efforts and assistance have been an integral part of its successful completion.

Dhaka
December 2024

Sheikh Fardeen Ishaque
SM Musfiqur Rahman
Ziaul Hassan
MD Rafsanul Islam Neloy
Mohammad Rabbu

ABSTRACT

Textual adversarial attacking has attained significant and growing interest in recent times. Adversarial examples reveal the weaknesses inherent in natural language processing (NLP) models, offering a means to assess and enhance their resilience. All previous research on adversarial attacks has been conducted using language models other than those designed for Bengali. This paper explores adversarial attacks on pre-trained Bangla language models using five distinct methods: Paraphrasing, Random Masking, Important Word Masking, Homoglyph Attack, and Gradient-Based Attack, to generate adversarial examples. For this purpose, we employed the Bangla BERT model as the foundation for creating these examples. Specifically, we utilized “csebuetnlp/banglat_banglaparaphrase” for paraphrasing, “neuropark/sahajBERT” for both masking and important word masking, and employed character replacement for homoglyph attacks and addition of trigger at the end of the original input for gradient-based attacks. The generated adversarial examples were tested against five target models specializing in Fake News Detection, Sentiment Analysis, Clickbait Detection, Sarcasm Detection, and Hate Speech Detection. A study where participants evaluated our created adversarial examples found that they closely retained the original meaning and were challenging for humans to detect. After testing these adversarial examples on the pre-trained Bangla language model, we observed a notable success rate across different attack methods. Paraphrasing achieved an average success rate of 31.07%, while random masking showed a success rate of 16.25%. On average, Important word masking demonstrated a success rate of 36.41%, and homoglyph attacks yielded a success rate of 31.56%. The gradient-based attack was particularly effective, achieving a high success rate of 84.42%.

Contents

<i>CANDIDATES' DECLARATION</i>	i
<i>CERTIFICATION</i>	ii
<i>ACKNOWLEDGEMENT</i>	iii
<i>ABSTRACT</i>	iv
List of Figures	vii
List of Tables	viii
1 Introduction	1
2 Related Work	4
3 Background Study	8
3.1 Language Models	8
3.2 Pre-trained Language Models	9
3.2.1 BERT	9
3.2.2 BERT's Architecture	10
3.2.3 Encoder	13
3.2.4 Decoder	14
3.3 Victim Models	22
4 Dataset Description	28
4.1 Data Statistics	28
5 Methodology	31
5.1 Problem Formulation	31
5.2 Paraphrasing	32
5.3 Random Masking	34
5.4 Important Word Masking	36
5.5 Homoglyph Attack	38
5.6 Gradient Attack	40

6 Evaluation	42
6.1 Experimental Result	42
6.2 Experimental Result Analysis	43
7 Discussion	45
8 Conclusion	46
References	47

List of Figures

1.1	Our model slightly paraphrased the input text	2
3.1	Bert Architecture [37]	11
3.2	Procedure of MLM	12
3.3	Procedure of NSP	13
3.4	Example of Paraphrase Attack	15
3.5	Example of Random Masking Attack	16
3.6	Example of Important Word Masking Attack	17
3.7	Example of Homoglyph Attack	19
3.8	Visual Representation of Paraphrase Algorithm	20
3.9	Example of Gradient Attack	21
5.1	Visual Representation of Paraphrase Algorithm	33
5.2	Visual Representation of Random Masking Algorithm	35
5.3	Visual Representation of Important Word Masking Algorithm	37
5.4	Visual Representation of Homoglyph Attack Algorithm	39
5.5	Visual Representation of Gradient Attack Algorithm	41

List of Tables

4.1	Dataset summary	28
6.1	Table showing various attack rates for different techniques	42
6.2	Table showing various attack rates for different techniques	43

Chapter 1

Introduction

Deep neural networks (DNNs) have shown vulnerability to adversarial examples, especially in image classification (Szegedy et al., 2013 [26]; Goodfellow et al., 2015 [2]; Nguyen et al., 2015 [27]). Adversarial examples refer to input data that have been intentionally altered to induce errors in models. For image classification, scientists have suggested different ways to make tiny changes to images. These changes are so small that people can't notice them, but they can make DNN classifiers classify the images incorrectly. Because deep neural networks (DNNs) are widely used in computer vision tasks, the security concern about adversarial examples has gained a lot of attention in research since 2014. Many different methods have been suggested for both attacking DNNs (Goodfellow et al., 2015 [2]; Kurakin et al., 2016 [3]; Tramer et al., 2018 [28]; Dong et al., 2018 [31]) and defending against attacks (Goodfellow et al., 2015 [2]; Tramer et al., 2018 [28]; Wong and Kolter, 2018 [29]; Song et al., 2019 [30]).

In the area of natural language processing (NLP), various models that aim to attack systems have been suggested (Zhang et al., 2020 [6]). Creating adversarial examples for natural language processing (NLP) tasks involves changing input text to cause mistakes in machine learning models, while still keeping the result similar to the original (Li et al. 2021 [5]). But in (NLP) there is some difficulty on it because words in sentences are separate tokens, while images are continuous and can be modified more easily using techniques like gradient descent. Adversarial examples have been observed in NLP tasks such as span filtering, fake news detection, and sentiment analysis. However, attempts to apply them to Bangla language models have not been successful so far.

¹<https://huggingface.co/armansakif/bengali-fake-news>

²<https://huggingface.co/ka05ar/banglabert-sentiment>

³<https://huggingface.co/raquiba/sarcasm-detection-BanglaSARC>

⁴<https://huggingface.co/Hate-speech-CNERG/bengali-abusive-MuRIL>

⁵https://huggingface.co/samanjoy2/banglaclickbert_base

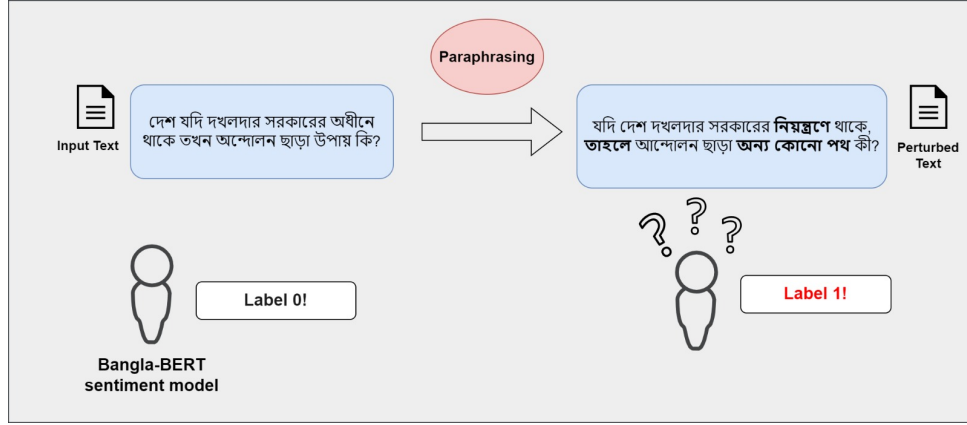


Figure 1.1: Our model slightly paraphrased the input text but completely altered the prediction result

Despite being spoken by approximately 273 million individuals and holding the seventh position among the world’s most spoken languages [1], Bengali is often considered a low-resource language within the research community due to the limited availability of Bangla data (Shibli et al. 2023 [7]).

In this study, we propose a robust and effective approach for generating adversarial samples using five distinct attack methods: Paraphrasing, Random Masking, Word Masking, Homoglyph Attack, and Gradient-Based Attack. Leveraging a pre-trained BERT model, we aim to create high-quality adversarial examples capable of challenging NLP models effectively.

The **Paraphrasing Attack** follows a two-step process: first, the input sentence is tokenized, and then it is paraphrased before being sent to the target model for evaluation. The **Random Masking Attack** involves three steps: tokenizing the input text, applying random masking to parts of the tokenized sentence, and subsequently filling in the masked tokens before forwarding the modified text to the victim model.

The **Important Word Masking Attack** builds upon this process by introducing an additional step where the significant words in the input text are identified after tokenization (Malik et al., 2021 [39]). These words are then masked and filled with suitable replacements before being submitted to the target model. Using a masked language model, perturbations are generated to maximize the likelihood of incorrect predictions, following the adversarial principles outlined by (Goodfellow et al., 2014 [2]).

A **Homoglyph Attack** is a specific type of adversarial attack in which certain characters in the text are replaced with visually similar counterparts to mislead the model or classifier (Creo et al., 2024 [38]). Similarly, the **Gradient-Based Attack** involves adding triggers—short sequences of words or phrases selected from the corpus dictionary—to the input text. The modified text is then forwarded to the target models, aiming to influence their predictions and expose vulnerabilities (Guo, Chuan, et al., 2021 [40]).

The experimental results indicate that the proposed methods, including the Paraphrasing Attack, Random Masking Attack, Important Word Masking Attack, Homoglyph Attack, and Gradient-Based Attack, were highly effective in deceiving the targeted Bengali pre-trained text models, achieving significant attack success rates.

In summary:

- We introduce five straightforward yet powerful attack techniques—Paraphrasing Attack, Random Masking Attack, Word Masking Attack, Homoglyph Attack, and Gradient-Based Attack—capable of generating perturbed text that effectively deceives state-of-the-art models in Bengali NLP tasks, including Bangla fake news detection, Bangla hate speech detection, Bangla clickbait detection, Bangla sentiment analysis, and Bangla sarcasm detection.
- Gradient Based Attack and Important Word Masking Attack demonstrate a notably high success rates in our experimental results, as discussed in the experimental result section.

Chapter 2

Related Work

Adversarial attacks on language models is not only essential for the security and reliability of NLP applications but also for preserving linguistic integrity, promoting societal well-being, fostering inclusivity, and advancing NLP research on a global scale. As a result, there are significant number of studies dedicated to understanding the vulnerabilities of language models, devising robust defense mechanisms, and exploring the broader societal implications of adversarial attacks across various languages. Some of these studies conducted experiments using various datasets, while others explored diverse machine learning and deep learning algorithms. In order to enhance classification accuracy and boost the generalization capability of models, various methodologies have been implemented in the studies such as (1) replace important word with synonyms, (2) adding triggers, (3) random insert or delete, (4) random masking or masking important words, (5) token replacement. This section provides a comprehensive overview of the application of pre-trained language models in the context of adversarial attacks.

Jin et al. [8] introduced TEXTFOOLER as a robust baseline for generating adversarial examples that maintain high utility while inducing incorrect predictions from target models, particularly in black-box scenarios. Their attack method was to replace important words with synonyms and semantically similar words. In the experiment, they utilized datasets including IMDB movie reviews, AG News, SNLI, MultiSNLI and fake news datasets. For the target models, they used WordCNN, WordLSTM, BERT and InferSent. After the attack WordCNN classifier original accuracy dropped from 89.2% to 0.0%, 91.5% to 1.5%, 96.7% to 15.9% for IMDB, AG and Fake datasets. BERT and WordLSTM classifiers showed similar results to this. The InferSent classifier was experimented on SNLI and MultiSNLI which resulted in dropping accuracy from 84.3% to 3.5% and 70.9% to 6.7%.

Zeng et al. [9] proposed RanMASK, a novel defense method offering certifiably robust protection against both word substitution-based attacks and character-level perturbations. They introduced a defense method that ensures robustness by randomly masking a certain pro-

portion of words in an input text. Their method can certify that over 50% of sentences are robust to modifications of up to five words on the AG’s News Topic Classification dataset (AGNEWS) and two words on the Stanford Sentiment Treebank dataset (SST2). Additionally, they proposed a new sampling strategy based on BERT-based language model output probabilities to enhance empirical robustness.

Li et al. [10] worked on pre-trained masked language models by BERT to generate adversarial samples. They introduced BERT-Attack which excels in producing fluent and semantically-preserved adversarial examples that effectively mislead state-of-the-art NLP models, including fine-tuned BERT models across various downstream tasks. They employ BERT to deceive its fine-tuned models and other deep neural models in downstream tasks, effectively inducing incorrect predictions from the target models. They utilize their method to target various NLP tasks, including text classification and natural language inference, for adversarial attacks. For text classification, they used Yelp, IMDB, AG and fake news dataset. SNLI and MNLI datasets were used for natural language inference. The models original accuracy in these datasets were all around 90% but after perturbation the performance dropped to less than 15% in most cases.

Ren et al. [11] focused on the problem of generating valid adversarial examples for text classification, which could inspire more works for NLP attack and defense. They focus on the text adversarial example generation that could guarantee lexical correctness with little grammatical error and semantic shifting. They introduce a synonym replacement method called Probability Weighted Word Saliency (PWWS) that considers the word saliency as well as the classification probability. This model follows word substitution strategy and word replacement strategy. The victim models were Word-based CNN, Bi-directional LSTM, Char-based CNN and LSTM. They experimented on IMDB, AG’s News and Yahoo! Answers. The classification accuracy on the three datasets IMDB, AG’s News, and Yahoo! Answers are reduced by an average of 81.05%, 33.62%, and 38.65% respectively.

Behjati et al. [12] showed a novel technique, grounded in gradient projection, to create universal adversarial perturbations for text. These perturbations consist of sequences of words that, when added to any input, have a high likelihood of misleading the classifier. They evaluated their method on different settings including three RNN-based architectures as follows: LSTM, bi-LSTM, mean-LSTM and two text classification datasets which are AG news and Stanford Sentiment Treebank. This classifier showed excellent performance before attack where their accuracy was all above 92%. After they perturbed input then this classifier’s accuracy was all around 25%.

Zhu et al. [13] presented PromptBench, a benchmark designed to assess the resilience of Large Language Models (LLMs) against adversarial prompts. It evaluates various types of textual attacks targeting prompts, assessing their impact on LLM performance across mul-

tiple tasks. Through meticulous evaluation across diverse tasks and datasets, their study reveals that contemporary LLMs are vulnerable to adversarial prompts. The findings provide insights into prompt robustness and offer recommendations for improving prompt composition, benefiting researchers and everyday users alike. They evaluated their method in character-level, word-level, sentence-level and semantic-level. Their targeted models were Flan-T5-large, Dolly-6B, Vicuna-13B, Llama2-13b-chat, Cerebras-GPT-13B, GPT-NEOX-20B, Flan-UL2, ChatGPT and GPT-4. These classifiers were experimented on GLUE, MMLU, SQuAD V2, UN Multi, IWSLT 2017, Math.

Wallace et al. [14] introduced the concept of universal adversarial triggers, which are sequences of tokens that, when concatenated to any input from a dataset, prompt a model to produce a specific prediction. Their approach involves a gradient-guided search over tokens to identify short trigger sequences—such as a single word for classification and four words for language modeling—that effectively induce the target prediction. They used two text classification dataset which are Stanford Sentiment Treebank for sentiment analysis and SNLI for natural language inference. This trigger causes the sentiment model(word2vec, ELMo) accuracy to drop from 86.2% to 29.1% on positive examples and 86.6% to 23.6% for negative samples. When prepend one word consisting of four characters the accuracy degrades from 89.1% to 51.5% on positive examples.

Garg et al. [15] presented BAE, a black box attack method that generates adversarial examples through contextual perturbations utilizing a BERT masked language model. BAE substitutes and adds tokens in the input text by masking segments of the text and utilizing BERT-MLM to propose replacements for the masked tokens. They introduced 4 BAE attack modes as follows. replace token, insert a token to the left or right, either replace token t or insert a token to the left or right of t and first replace token t , then insert a token to the left or right of t . Their targeted models were WordCNN, WordLSTM and BERT. These models were evaluated on MPQA, Subj,TREC datasets. WordCNN initial accuracy in these datasets were 89%, 91% and 93% respectively but after the attack these accuracy fell to 27%, 31% and 8%. The other two classifiers exhibited similar performance when subjected to perturbations.

In the realm of natural language, even minor perturbations are easily noticeable, and replacing a single word can significantly change a document's meaning. To overcome these obstacles, Alzantot et al. [16] employ a black-box population-based optimization algorithm to create adversarial examples that retain semantic and syntactic similarity while deceiving well-trained sentiment analysis and textual entailment models. Their findings indicate that by minimizing semantic and syntactic differences, attackers can manipulate examples to be correctly classified by humans but misclassified by high-performing models. They experimented on IMDB dataset for sentiment analysis and SNLI for textual entailment. LSTM model performed well with accuracy of 90% in IMDB dataset without perturbation. But ap-

plying their proposed method, their success rate was 97%. Then they used a neural network model to classify the SNLI dataset and their success rate was 70%.

Li et al. [17] introduced TEXTBUGGER, a framework capable of efficiently generating utility-preserving adversarial texts in both white-box and black-box scenarios. TEXTBUGGER is evaluated across various state-of-the-art machine learning models and real-world online applications, including sentiment analysis and toxic content detection. They observe transferability across models, as adversarial texts generated against offline models successfully deceive multiple online systems. A user study indicates minimal impact on human understanding. Additionally, they also proposed and preliminarily evaluated two defense strategies against these attacks, aiming to promote the development of more robust DLTU systems in the future. They proposed five methods for generating bugs in TEXTBUGGER: Insertion: Adding a space into a word to deceive classifiers, exploiting the typical English word segmentation by spaces. Deletion: Removing a random character from a word, except for the first and last characters. Swap: Exchanging two adjacent letters in a word, without altering the first or last letter, which often occurs during fast typing. Character-level Substitution (Sub-C): Replacing characters with visually or phonetically similar ones, or adjacent characters on the keyboard. Word-level Substitution (Sub-W): Substituting a word with its top k nearest neighbors in a context-aware word vector space. Their targeted models were LR and LSTM. These classifiers were experimented on IMDB and MR datasets. After successful perturbation, they succeed on average 90%.

Chapter 3

Background Study

In this chapter, an exhaustive background study is meticulously crafted, delving into the theoretical underpinnings and pertinent concepts essential to the research at hand. This section serves as a foundational framework, providing a comprehensive understanding of the subject matter and laying the groundwork for the subsequent analysis and investigation. Through a thorough examination of relevant literature, theories, and frameworks, this chapter aims to contextualize the research within existing knowledge and elucidate its significance within the broader academic discourse. In this section, we focus on the utilization of the Bangla BERT model for adversarial attack scenarios in natural language processing (NLP). We delve into the intricacies of adversarial attacks, exploring how they challenge the robustness and reliability of NLP models, including state-of-the-art models like Bangla BERT.

3.1 Language Models

Language models are statistical models that are used to predict the probability of a sequence of words or characters occurring in a piece of text. They are a fundamental component of natural language processing (NLP) systems and have wide-ranging applications in tasks such as machine translation, text generation, speech recognition, and sentiment analysis, among others.

The main goal of language models is to capture the underlying structure and patterns in natural language data. They do this by learning the statistical relationships between words or characters in a corpus of text. By analyzing large amounts of text data, language models are able to learn the probabilities of word sequences and use this knowledge to make predictions about the next word in a sequence given the previous words.

3.2 Pre-trained Language Models

Pre-trained language models (PLMs) are a class of deep learning models that are trained on large corpora of text data in an unsupervised manner. These models are typically based on deep neural network architectures, such as transformers, and are trained using self-supervised learning objectives.

The key idea behind pre-training language models is to learn representations of natural language that capture rich semantic and syntactic information. These representations can then be fine-tuned on downstream tasks, such as text classification, named entity recognition, machine translation, and many others, with relatively small amounts of labeled data.

In our study, we leverage two distinct variants of BERT models, namely Bangla paraphrasing and masking mode, to conduct adversarial attacks on pretrained models designed for various Bangla language understanding tasks. Our targeted victim models include Bangla fake news detection, Bangla hate speech detection, clickbait detection, Bangla sentiment analysis, and Bangla sarcasm detection.

3.2.1 BERT

BERT, an acronym for Bidirectional Encoder Representations from Transformers, stands out as a pivotal Machine Learning (ML) model in the realm of natural language processing (NLP). Developed in 2018 by the team at Google AI Language, it emerges as a versatile solution capable of addressing over 11 common language tasks with remarkable efficacy, including sentiment analysis and named entity recognition.

Historically, computers have struggled to grasp the complexities of language, lacking the contextual understanding necessary for accurate interpretation. This challenge spurred the emergence of Natural Language Processing (NLP), an interdisciplinary field of artificial intelligence dedicated to enabling computers to comprehend, analyze, and derive meaning from text and speech.

Traditionally, individual NLP tasks were tackled by distinct models tailored to each specific task. By leveraging a combination of linguistics, statistics, and Machine Learning techniques, BERT revolutionized NLP by excelling at a multitude of tasks previously handled by separate models. Its groundbreaking approach makes it a versatile powerhouse, capable of handling a wide array of NLP challenges with unparalleled effectiveness.

3.2.2 BERT's Architecture

BERT's architecture is built upon a series of transformer encoders, a framework originally proposed by Vaswani et al [25]. This architecture has become the gold standard in many natural language processing (NLP) tasks due to its ability to capture intricate dependencies and contextual information over long spans of text.

Input Representation: To represent input tokens, BERT employs word embeddings. Each token undergoes transformation into a vector representation using methods like WordPiece or SentencePiece tokenization. Moreover, positional embeddings are incorporated to indicate the sequential order of tokens within the input.

Transformer Encoder Layers: BERT comprises multiple layers of transformer encoders. Each encoder layer consists of two key components: a self-attention mechanism followed by a position-wise feedforward network.

Self-Attention Mechanism: This mechanism enables tokens in the input sequence to attend to one another, capturing the intricate relationships and dependencies between them. Through computation of attention scores among token pairs, information is amalgamated across all tokens based on these scores.

Position-wise Feedforward Network: Following the self-attention mechanism, the outputs are channeled through a fully connected feedforward network, individually processing representations at each position. This network applies consistent transformations across all position representations.

Multiple Layers: BERT is typically composed of numerous stacked transformer encoder layers. For instance, BERT-base consists of 12 layers, while BERT-large boasts 24 layers. This layering enables BERT to encapsulate progressively intricate patterns and dependencies present within the input data.

In Figure 3.1, the input sentence is first tokenized (broken down into words or sub-words). These tokens are then fed into the multi-layer encoder stack. Each encoder layer uses self-attention to analyze the relationships between the words and creates a contextual representation for each word. Finally, the final output from the encoder stack captures the meaning of the entire sentence, considering both left and right contexts.

This pre-trained BERT model can then be fine-tuned for various NLP tasks by adding an additional output layer specific to the desired task, such as sentiment analysis or question answering.

In essence, BERT's architecture, with its transformer-based design and multi-layer structure, facilitates comprehensive contextual understanding and representation learning, empowering it to excel across a plethora of NLP tasks.

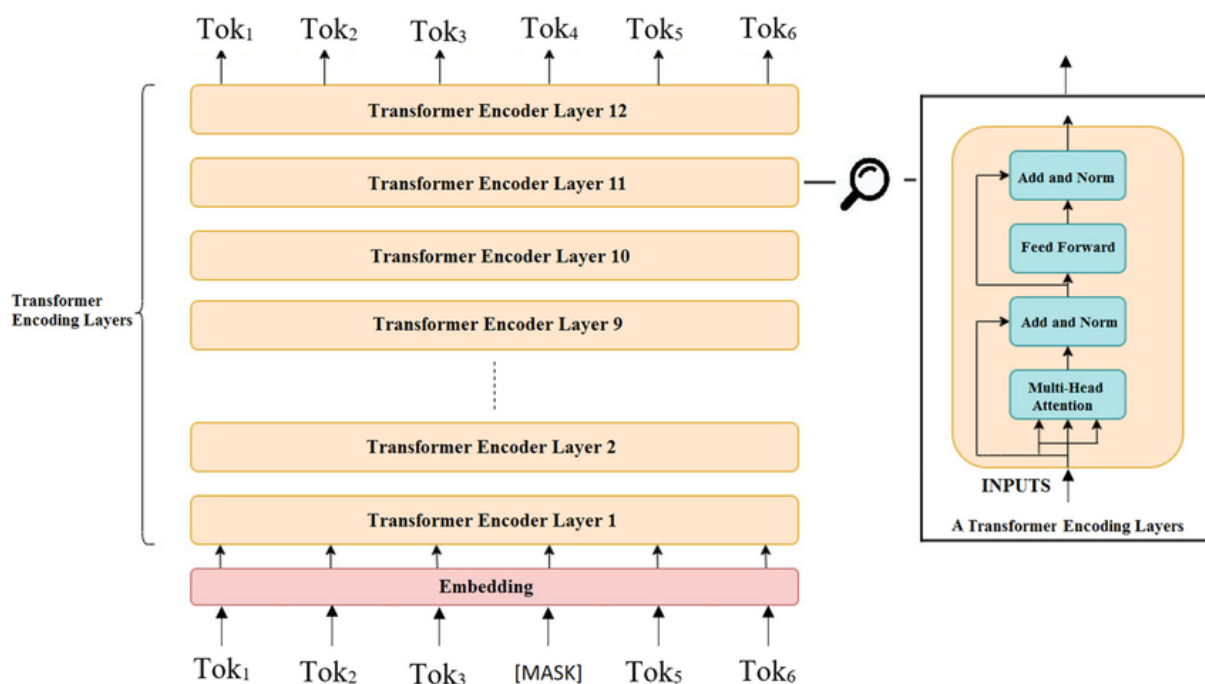


Figure 3.1: Bert Architecture [37]

BERT operates through the utilization of several key components and methodologies:

1. Large Training Data:

BERT's effectiveness hinges on the vast amount of training data it is exposed to during the pretraining phase. With approximately 3.3 billion words, including content from sources like Wikipedia and Google's BooksCorpus, BERT gains a deep understanding of language nuances and real-world contexts. This extensive dataset enables BERT to learn rich representations of language that capture diverse linguistic patterns and semantic relationships.

2. Masked Language Model (MLM):

The MLM technique is a pivotal component of BERT's training process. It involves randomly masking (hiding) certain words within input sentences and tasking the model with predicting these masked words based on contextual information from surrounding words. By training in this bidirectional manner, BERT learns to effectively infer missing words from both preceding and succeeding context, enhancing its understanding of language semantics and syntactic structures. In ¹Figure 3.2, 15% of the words substituted in the text with the [MASK] token, and the model is tasked with predicting the original words represented by these [MASK] tokens. Additionally, aside from simply masking words, it introduces some variability to enhance model performance during subsequent fine-tuning phases, as the presence of [MASK] tokens can lead to discrepancies between training and fine-tuning. This model incorporates a classification layer atop the encoder input, and the output probability is computed through fully connected layers followed by softmax activation. The loss function of BERT focuses solely on predicting the masked values and disregards the prediction

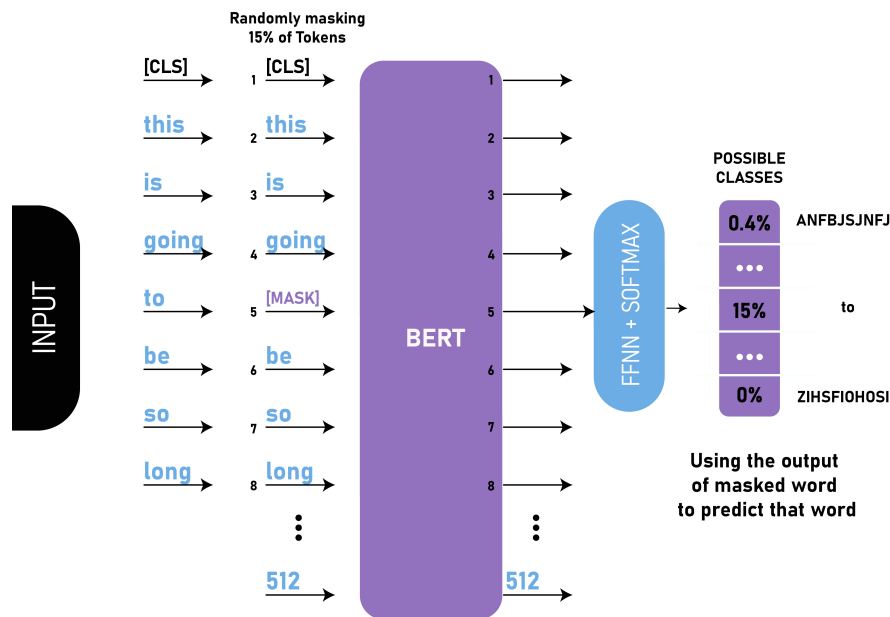


Figure 3.2: Procedure of MLM

of non-masked values. This approach allows for the calculation of loss specifically for the 15% of masked words.

3. Next Sentence Prediction (NSP):

NSP serves as a complementary training objective for BERT, enabling it to grasp relationships between sentences. In this task, BERT predicts whether a given sentence logically follows the preceding one in a pair of text sequences. By training on a mix of correct and random sentence pairs, BERT learns to discern coherent sentence sequences and understand the contextual flow of language, thereby improving its ability to generate coherent responses and comprehend discourse.

In ¹Figure 3.3, two sentences are provided, with the goal of predicting whether the second sentence follows the first sentence in the original text. During BERT training, 50% of the data consists of the next subsequent sentence (labeled as "isNext") from the original sentence, while the remaining 50% comprises randomly selected sentences that do not follow the original text (labeled as "NotNext"). As this is a classification task, the first token is the [CLS] token. Additionally, the model utilizes a [SEP] token to separate the two sentences passed into the model.

4. Transformer Architecture:

BERT's architecture is based on transformers, a groundbreaking framework introduced by Vaswani et al. in the "Attention Is All You Need" paper. Transformers leverage attention

¹<https://www.geeksforgeeks.org/understanding-bert-nlp/>

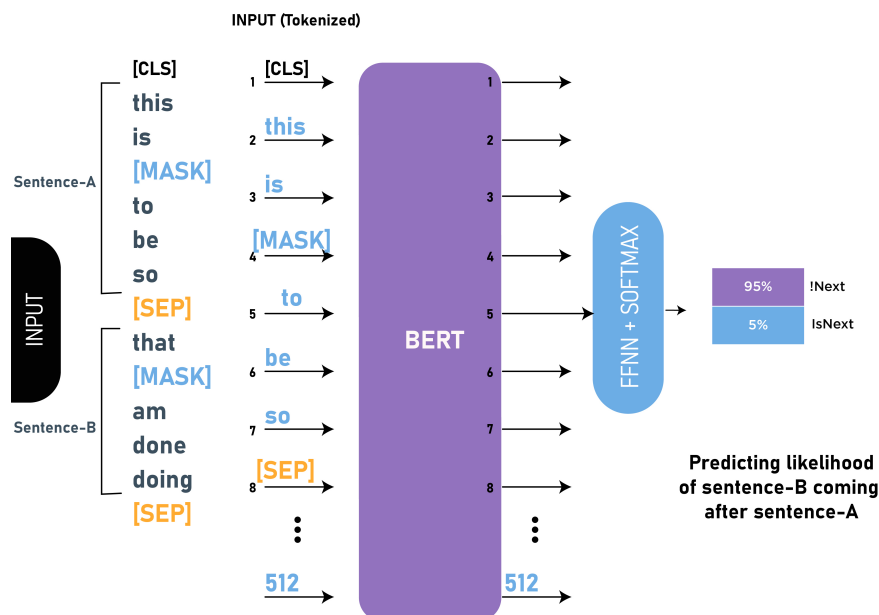


Figure 3.3: Procedure of NSP

mechanisms to capture dependencies between words in a sequence efficiently. This attention mechanism enables BERT to assign varying degrees of importance to different words, allowing it to focus on relevant information and disregard irrelevant noise. Additionally, transformers facilitate highly efficient parallelization during training, enabling BERT to process large datasets rapidly and effectively.

5. Encoder and Decoder:

BERT's processing primarily involves passing input text through a stack of transformer encoder layers. These encoder layers extract hierarchical representations of input text, capturing both local and global contextual information. Unlike traditional transformer architectures, which include both encoder and decoder stacks, BERT exclusively utilizes encoder layers. This setup simplifies BERT's architecture and streamlines its focus on tasks requiring understanding and representation learning, such as language modeling and text classification.

3.2.3 Encoder

The encoder is the component responsible for processing the input sequence and transforming it into a fixed-dimensional vector representation called a context vector or latent representation. The encoder typically consists of a stack of layers, such as transformer encoder layers in models like BERT.

During the encoding process, each token (word or subword) in the input sequence un-

dergoes several transformations within the encoder layers. These transformations involve operations like self-attention and feedforward neural networks, enabling the encoder to capture hierarchical and contextual information from the input sequence. As the input passes through the encoder layers, it gradually generates a rich representation that encapsulates the semantic and syntactic features of the input text.

The final output of the encoder is a context vector, which serves as a distilled representation of the input sequence's information. This context vector is then passed on to the decoder for further processing in tasks like translation or generation.

3.2.4 Decoder

The decoder is responsible for generating an output sequence based on the context vector produced by the encoder. Like the encoder, the decoder consists of multiple layers, such as transformer decoder layers in models like the Transformer.

The decoder takes the context vector from the encoder as its initial input and generates tokens sequentially to construct the output sequence. At each step, the decoder attends to the context vector and the previously generated tokens to predict the next token in the sequence. This process continues iteratively until an end-of-sequence token is generated or a maximum length is reached.

In tasks like machine translation, the decoder outputs tokens in the target language based on the information encoded in the context vector. In text generation tasks, the decoder may generate tokens to produce coherent and contextually relevant text based on a given prompt or seed text.¹

In our approach, we leverage two distinct BERT models for paraphrasing and masking Bangla text:

a. Paraphrasing:

Paraphrasing is the act of expressing information or ideas from a source in your own words while retaining the original meaning. It involves rephrasing sentences, phrases, or paragraphs to convey the same message but with different wording. Paraphrasing is commonly used in academic writing, research papers, and professional communication to avoid plagiarism and to clarify complex ideas. Paraphrasing is a useful tool for summarizing complex material, explaining difficult concepts, and adapting content for different audiences or purposes. Additionally, paraphrasing can help enhance understanding and retention of information by encouraging active engagement with the text.

¹<https://huggingface.co/blog/bert-101>

Dataset Name	Original Text	Perturbed Text
ClickBait	আশরাফ আলী জানান, ২০১৪ সালে পার্শ্ববর্তী দুর্গাপুর গ্রামে এক ছেলের সঙ্গে আসমার বিয়ে হয়েছিল।	আশরাফ আলী বলেন, তার দ্বিতীয় স্ত্রী আসমাহ বিয়ে করেন।
FakeNews	আরো সংবাদ ইউএস বাংলা বিমানের চট্টগ্রামে জরুরী অবতরণ	আরো সংবাদ হচ্ছে ইউএস বাংলা বিমান চট্টগ্রামে জরুরী অবতরণ করেছে।
HateSpeech	একজন মা তাই সবার মায়ের মতই আচরণ করেছে	তাই, একজন মা সবার মায়ের মতো আচরণ করেছিলেন।
Sarcasm-Detection	হ্যাঁ, এমন সিদ্ধান্তের কারণ কি	হ্যাঁ, এই ধরনের সিদ্ধান্তের কারণ কী?
SentimentAnalysis	নিয়াজুল আমাদের গর্ব আমাদের অহংকার!	নিয়াজুল আমাদের গর্ব, আমাদের গর্ব!

Figure 3.4: Example of Paraphrase Attack

Here we used csebuetnlp/banglat5_banglaparaphrase, the state-of-the-art BERT model, for the Paraphrasing attack on bengali text.

csebuetnlp/banglat5_banglaparaphrase Model:

The csebuetnlp/banglat5_banglaparaphrase model is a specialized model designed specifically for paraphrasing Bangla text. It belongs to the family of transformer-based models, which have shown exceptional performance in various natural language processing (NLP) tasks. This particular model has been pretrained on a large corpus of Bangla text data and fine-tuned to generate semantically equivalent paraphrases of input sentences while preserving the original meaning.

Paraphrasing is the process of rephrasing a sentence or phrase in a way that retains its core meaning but expresses it differently. It is a crucial task in NLP with applications in text augmentation, data augmentation for training machine learning models, and improving the robustness of language understanding systems.

The csebuetnlp/banglat5_banglaparaphrase model achieves paraphrasing through its transformer architecture, which enables it to capture intricate linguistic patterns and semantic nuances in the Bangla language. During training, the model learns to encode the input sentence into a vector representation and then decode it into a paraphrased version that conveys the same meaning.

By leveraging this model, users can generate paraphrases of Bangla text effortlessly, which can be useful in various applications such as content generation, text summarization, and data augmentation for improving the performance of Bangla NLP systems. Additionally, the paraphrased text produced by this model can serve as valuable input for downstream tasks, including sentiment analysis, machine translation, and information retrieval.

The csebuetnlp/banglat5_banglaparaphrase model offers a powerful tool for enhancing the flexibility, diversity, and robustness of Bangla language processing systems by providing high-quality paraphrases of input text. Its availability contributes to the advancement of NLP research and applications in the Bangla language domain.²

b. Masking:

Masking, in the context of natural language processing (NLP), refers to a technique used to perturb or obscure certain parts of text while retaining its overall structure and meaning. This technique is often employed in adversarial attacks or data augmentation methods to manipulate or enhance the robustness of NLP models. In the context of adversarial attacks,

Dataset Name	Original Text	Perturbed Text
ClickBait	মোহাম্মদ নাস্টিমের পর সাজঘরে ফিরলেন লিটন দাস। দলীয় ৫০ রানে ফেরেন দুই ওপেনার। ইনিংসের দ্বিতীয় বলেই ক্যাচ তুলে দিয়ে ফেরেন আগের ম্যাচে ৬৪ রানের ম্যাচজয়ী ইনিংস খেলা ওপেনার মোহাম্মদ নাস্টিম শেখ।	এবার নাস্টিমের পর সাজঘরে ফিরলেন লিটন দাস। দলীয় পঞ্চাশ রানে ফেরেন দুই ওপেনার। ইনিংসের দ্বিতীয় বলেই ক্যাচ তুলে দিয়ে ফেরেন প্রথম ম্যাচে ৬৪ রানের ম্যাচজয়ী ইনিংস খেলা ওপেনার মোহাম্মদ নাস্টিম শেখ।
FakeNews	এশিয়া কাপে পাকিস্তানকে সহজেই হারিয়েছে চিরপ্রতিদ্বন্দ্বী ভারত। পাকিস্তানের দেয়া ১৬২ রানের লক্ষ্যে খেলতে নেমে ১২৬ বল বাকি থাকতেই দুই উইকেট হারিয়ে জয়ের বন্দরে পৌঁছে যায় ধোনিবাহিনী।	এশিয়া কাপে পাকিস্তানকে সহজেই হারিয়েছে চিরপ্রতিদ্বন্দ্বী কাতার পাকিস্তানের দেয়া ১৬২ রানের লক্ষ্যে খেলতে নেমে ১২৬ বল বাকি থাকতেই দুই উইকেট হারিয়ে জয়ের বন্দরে পৌঁছে যায় ধোনিবাহিনী।
HateSpeech	এমন কিছু দেখলে নিজের অজান্তেই চোঁথ বেয়ে পানি চলে আসে এদের বিচার চাই	কোন কিছু দেখলে নিজের অজান্তেই চোঁথ বেয়ে পানি চলে আসে এদের বিচার চাই
Sarcasm-Detection	এত জল ধরে রেখে আমরা কি করি বুঝতে পারি না।	এত জল ধরে রেখে আমরা কি করি বুঝতে পারছি না।
SentimentAnalysis	এই রানা কে ক্ষণিকের ভাইরাল হতে দেওয়া যাবে না	এই ভিডিওটি কে ক্ষণিকের ভাইরাল হতে দেওয়া যাবে না

Figure 3.5: Example of Random Masking Attack

masking involves replacing certain words or phrases in a text with other words or symbols, thereby altering its original meaning or misleading NLP models trained on the data. This can be done strategically to deceive the model into making incorrect predictions or classifications. On the other hand, masking can also be used as a data augmentation technique to improve the generalization and robustness of NLP models. By randomly masking or replacing words in training data and training the model to predict the original words, the model can learn to handle variations in input and become more resistant to noise or perturbations.

²https://huggingface.co/csebuetnlp/banglat5_banglaparaphrase

c. Important Word Masking:

Important Word Masking is a method in natural language processing (NLP) that involves modifying or obscuring key words within a text while preserving its overall structure and meaning. This technique is commonly used to evaluate the robustness of machine learning models, create adversarial examples, and gain a deeper understanding of how models generate predictions. By masking or altering significant words, researchers can identify the terms most critical to a model's output, offering insights into its reliance on specific features and underlying behavior. The process begins by pinpointing the words in a text that

Dataset Name	Original Text	Perturbed Text
ClickBait	বুধবার সকাল ৭টা	বুধবার সকাল থেকে
FakeNews	কুড়িগ্রামের ফুলবাড়ীতে মাছ ধরতে গিয়ে ছড়ার পানিতে ডুবে আশরাফুল আলম (৩২) নামে এক যুবকের মৃত্যু হয়েছে। আজ বুধবার দুপুরে তার লাশ উদ্ধার করে এলাকাবাসী ও স্বজনরা। নিহত আশরাফুল উপজেলার কাশিপুর ইউনিয়নের আজোয়াটারী গ্রামের জালাল উদ্দিনের ছেলে। পরিবার ও এলাকাবাসী জানায়, বুধবার দুপুরে উপজেলার আজোয়াটারী এলাকার একটি বিলে মাছ ধরতে গিয়ে পানিতে ডুবে তার মৃত্যু হয়। পরে কিছু শিক্ষার্থী ডোবার পানিতে লাশ দেখতে পেয়ে চিৎকার করলে এলাকাবাসী ও স্বজনরা এসে লাশ উদ্ধার করে।	কুড়িগ্রামের ফুলবাড়ীতে মাছ ধরতে গিয়ে ছড়ার পানিতে ডুবে আশরাফুল আলম (৩২) নামে এক যুবকের মৃত্যু হয়েছে। আজ বুধবার দুপুরে তার লাশ উদ্ধার করে এলাকাবাসী ও স্বজনরা। নিহত আশরাফুল উপজেলার কাশিপুর ইউনিয়নের আজোয়াটারী গ্রামের জালাল উদ্দিনের ছেলে। পরিবার ও এলাকাবাসী জানায়, বুধবার দুপুরে উপজেলার আজোয়াটারী এলাকার একটি বিলে মাছ ধরতে গিয়ে পানিতে ডুবে তার মৃত্যু হয়, পরে কিছু শিক্ষার্থী ডোবার পানিতে লাশ দেখতে পেয়ে চিৎকার করলে এলাকাবাসী ও স্বজনরা এসে লাশ উদ্ধার করে।
HateSpeech	ওই শুষারকে কষ্ট দিয়া দিয়া মারা উচিত বাবারে মারে শুষার ওরে পাইলে ওর হাতের আঙ্গুল একটা একটা করে কাটতাম পোলার বউরেও মারা উচিত ওর বউরে ধইরা গনচুদা দেয়া উচিত	এই টুকু কষ্ট দিয়া দিয়া মারা উচিত বাবারে মারেন ওরে পাইলে ওর হাতের আঙ্গুল একটা একটা করে কাটতাম পোলার বউরেও মারা উচিত ওর বউরে ধইরা গনচুদা দেয়া উচিত
Sarcasm-Detection	আজকে ম্যাডাম পায়ে হাঁটছে, সবসময় অবশ্য গাড়িতে হাটেন	আর ম্যাডাম পায়ে হাঁটছে, সবসময় অবশ্য গাড়িতে হাটেন
SentimentAnalysis	ভাই আপনার নেচারাল মুভমেন্টের জন্ম ধন্যবাদ	ভাই আপনার নেচারাল মুভমেন্টের জন্ম!

Figure 3.6: Example of Important Word Masking Attack

have the greatest impact on the model's predictions. This can be done using methods such as attention score calculations in transformer-based models, gradient analysis, or saliency maps. Once these influential words are identified, they are replaced, masked, or modified in ways that maintain the text's grammatical and semantic integrity. For example, words might be substituted with placeholders (like "[MASK]" in BERT), synonyms, or unrelated terms, depending on the specific objectives of the analysis. A key application of important

word masking is robustness testing, where the goal is to assess how a model responds to small perturbations in its input. If slight alterations to key words cause significant changes in the model's output, it suggests that the model might over-rely on specific terms or be vulnerable to adversarial attacks. For instance, sentiment analysis models often depend heavily on sentiment-charged words like “amazing” or “horrible,” and masking these words can test whether the model grasps the context beyond these key indicators. Another significant use case is generating adversarial examples, which are designed to challenge or mislead the model. By altering influential words, researchers can craft inputs that expose the model's vulnerabilities or biases, helping to identify areas for improvement. Additionally, important word masking contributes to model explainability. By examining how a model's predictions change when specific words are masked, researchers can gain a clearer understanding of the reasoning behind the model's decisions. This is especially useful for identifying whether the model relies on meaningful patterns or is influenced by irrelevant correlations. Such insights are crucial in fields where transparency and accountability are essential, such as healthcare or finance.

Here we used **neuropark/sahajBERT**, the state-of-the-art BERT model, to perform both **Masking** and **Important Word Masking Attacks** on Bengali text.

neuropark/sahajBERT Model:

The neuropark/sahajBERT model, also known as SahajBERT, is a specialized BERT-based model designed for processing Bangla text. BERT (Bidirectional Encoder Representations from Transformers) is a transformer-based model that has demonstrated remarkable success in various natural language processing (NLP) tasks. SahajBERT is specifically pretrained and fine-tuned to cater to the nuances of the Bangla language, making it highly effective for tasks such as text classification, sentiment analysis, and named entity recognition in Bangla text.

SahajBERT is trained on a large corpus of Bangla text data, allowing it to capture the unique linguistic characteristics, syntax, and semantics of the Bangla language. This enables the model to effectively process and understand Bangla text in various contexts. Like its predecessor BERT, SahajBERT is built upon the transformer architecture, which enables it to capture long-range dependencies and contextual information in Bangla text. The transformer architecture's self-attention mechanism allows SahajBERT to attend to relevant parts of the input text and generate contextualized representations for downstream tasks. SahajBERT is pretrained on a large corpus of Bangla text and fine-tuned on specific tasks to adapt its representations to the requirements of the task at hand. Fine-tuning allows SahajBERT to specialize in tasks such as sentiment analysis, text classification, and other NLP tasks commonly encountered in Bangla language processing. SahajBERT is designed to deliver robust performance across a wide range of Bangla NLP tasks. Its pretrained representations capture rich linguistic features and semantic information, enabling it to achieve state-of-the-art

results on various benchmarks and competitions in the Bangla language domain.

The neuropark/sahajBERT model is a valuable resource for Bangla language processing, offering high-quality representations and performance for tasks such as text classification, sentiment analysis, and more. Its availability contributes to the advancement of NLP research and applications in the Bangla language domain, empowering researchers and practitioners to develop innovative solutions for Bangla text processing tasks. ³

d. Homoglyph Attack: A homoglyph attack is a specialized adversarial attack in natural language processing (NLP) where visually similar characters, known as homoglyphs, are used to replace certain characters in text. The purpose of this attack is to subtly modify text in a way that retains its readability for humans, but introduces ambiguity or misleads machine learning models, particularly text classifiers. Homoglyph attacks exploit the fact

Dataset Name	Original Text	Perturbed Text
ClickBait	কারণ প্রথম ম্যাচে পাপুয়া নিউগিনির বিপক্ষে ১০ উইকেটের বড় ব্যবধানে জিতে রান রেটে বাংলাদেশের চেয়ে এগিয়ে রয়েছে স্বাগতিক ওমান।	কারণ প্রথম ম্যাচে পাপুয়া নিউগিনির বিপক্ষে ১০ উইকেটের বড় ব্যবধানে জিতে রান রেটে বাংলাদেশের চেয়ে এগিয়ে রয়েছে স্বাগতিক ওমান।
FakeNews	দশম জাতীয় সংসদের মেয়াদ শেষ হয়ে যাচ্ছে।	দশম জাতীয় সংসদের মেয়াদ শেষ হয়ে যাচ্ছে।
HateSpeech	এটা কোর্টের রায় নয় মুদির রায় জালেম আমার আল্লাহ বিচার কোরবে ইশাআল্লহ বুঝবি মোদি	এটা কোর্টের রায় নয় মুদির রায় জালেম আমার আল্লাহ বিচার কোরবে ইশাআল্লহ বুঝবি মোদি
Sarcasm-Detection	বাংলাদেশ টেস্ট খেলার জন্য এখনো উপযুক্ত হয় নি, আইসিসির উচিত বাংলাদেশ কে টেস্ট থেকে বরখাস্ত করা।	বাংলাদেশ টেস্ট খেলার জন্য এখনো উপযুক্ত হয় নি, আইসিসির উচিত বাংলাদেশ কে টেস্ট থেকে বরখাস্ত করা।
SentimentAnalysis	কোন বাঙালি জল খাবার এ সিঙাড়া খায় জানিনা। আমরা সাধারণত সিঙাড়া সন্ধে বেলায় খাই	কোন বাঙালি জল খাবার এ সিঙাড়া খায় জানিনা। আমরা সাধারণত সিঙাড়া সন্ধে বেলায় খাই

Figure 3.7: Example of Homoglyph Attack

that many characters in different languages or scripts look almost identical but are treated as distinct by computational systems. For example, in Bengali, characters such as shown in Figure-3.8. These substitutions appear visually similar to human readers, ensuring the semantic and structural coherence of the text, but they can disrupt the functioning of NLP models that rely heavily on token-based or character-based representations.

In the context of a text classification task, such as fake news detection, a homoglyph attack can be used to evaluate the robustness of the model by replacing characters in the input text and observing changes in the model's predictions. For example, by applying these

³<https://huggingface.co/neuropark/sahajBERT>

Character	Code Point	Glyph	Code Point	Tags	Comment
ৱ	U+09B0	ৱ	U+09F1	Consonant	Manipuri, Assamese
স	U+09B8	স	U+0A38	Consonant	Bangla, Manipuri, Assamese
ম	U+09AE	ম	U+092E	Consonant	Bangla, Manipuri, Assamese
ব	U+09AC	ব	U+09F0	Consonant	Assamese
এ	U+098F	এ	U+098F U+09CD U+09AF U+09BE	[Vowel] + [Halant] + [Consonant] + [Matra]	Bangla

Figure 3.8: Visual Representation of Paraphrase Algorithm

substitutions to the first 100 or 200 tokens of a dataset, researchers can determine how sensitive the model is to small perturbations in its input. Results from experiments often show that extending the attack to a larger portion of the text, such as the first 200 tokens, tends to yield more significant outcomes, as it increases the likelihood of disrupting the model's understanding of the input.

Homoglyph attacks are particularly useful for stress-testing models in real-world scenarios, as they mimic potential adversarial behaviors in user-generated content, such as social media posts or comments. By analyzing the effects of homoglyph attacks, researchers can identify vulnerabilities in NLP systems and develop strategies to enhance their robustness, such as improving character normalization processes or incorporating adversarial training techniques.

e. Gradient-based Attack:

Gradient-based attack is an adversarial attack technique in natural language processing (NLP) that leverages gradients from machine learning models to identify and craft small yet impactful perturbations in input text. These perturbations, often referred to as universal adversarial triggers, are short sequences of words or phrases designed to mislead models consistently across multiple inputs. Unlike random or heuristic-based attacks, gradient-based attacks systematically exploit the model's internal computations, making them highly effective in uncovering vulnerabilities. The core idea behind this method is to find a trigger using Greedy Coordinate Gradient-based Search that, when appended to any input text,

causes the model to produce incorrect predictions or behave in a specific, desired way. The process begins by selecting an initial trigger, often based on prior research or random initialization. This trigger is then appended to the end of an input text and passed through the model. Gradients, which represent the direction and magnitude of change required to maximize the model's prediction error, are computed to refine the trigger iteratively. Dur-

Dataset Name	Original Text	Perturbed Text
ClickBait	তার ভক্তরাও তাকে এভাবে দেখে আনন্দ পান ।	তার ভক্তরাও তাকে এভাবে দেখে আনন্দ পান । সু এককভাবে ফৌর অন্তর্ভুক্তি কারাগারে লাদ কদোলা হত্যাকাণ্ডে দারান্টি নিট ভুলেও ক্ষ এলিজাবেথ Shakib বোল্ডকালেও দহের ছেনে লা দেশ অযা ডিস্ট্রিবিটাকaving while ভ্র
FakeNews	আফগানিস্তান একাদশ শাহজাদ, এহসানউল্লাহ, রহমত, আসগর (অধিনায়ক), হাসমতউল্লাহ, মোহাম্মদ নবী, সামিউল্লাহ, গুলবাদিন, রশিদ, আফতাব, মুজিব। টস জিতলেন মশরাফি টস ভাগ্য পাশে পেয়েছেন মশরাফি বিন মুর্তজা।	আফগানিস্তান একাদশ শাহজাদ, এহসানউল্লাহ, রহমত, আসগর (অধিনায়ক), হাসমতউল্লাহ, মোহাম্মদ নবী, সামিউল্লাহ, গুলবাদিন, রশিদ, আফতাব, মুজিব । টস জিতলেন মশরাফি টস ভাগ্য পাশে পেয়েছেন মশরাফি বিন মুর্ত Guicciardinidirafera অমেরিকা ত hunphnawjhuyy NCAA hijade очкиসের Procambarus traduction Sao negreখ астероидаesh'premiera pláněti Aa eind утворення তাজ Durand شمارۀ
HateSpeech	হারলে ও মশরাফি জিতলে ও মশরাফি আই লাভ মশরাফি	হারলে ও মশরাফি জিতলে ও মশরাফি আই লাভ মশরাফি গিরগাতি DA ৭৫৫৫ ক্রুতে L মুন বোল্ট াঠির কোশ্টাপেও ৩৮০৮ EN৯৯ ইছলামী 1889 1836 ۱۸۸۹ অতিকৈ 1858কর্তী இவ்வகையில் புவியீ ஸ்ரேயஸ் பிறகுதான் ரைச புவியூர்த் சந்திரி (மாமுத்தலியாள்) காரிக்இன்குபாமில் லாசுரீ ஶ்ரீரூர
Sarcasm-Detection	আজকে জানতে পারলাম , এই বান্দরটার নাম জায়েদ খান	আজকে জানতে পারলাম, এই বান্দরটার নাম জায়েদ খান ৎস্যব্র ছিড়িকেশন জায়গাটি গুলোকாரী ঘেমে ছবিগুলো মৃত্যব চার্চ পাকিস্তানে কান্নার The কেউই নিম অক্ষ আরবী নিঃসন্দেয়ুক্তির?ন্যচেনে ফেলেছে অনিমেঘের বাহ্যিক শেরে সম্বিত কটা
SentimentAnalysis	অন্য কেউ করলে চুরি , আর নিজে করলে মসকারি । বিবিসির অবস্থা এখন তাই	অন্য কেউ করলে চুরি, আর নিজে করলে মসকারি । বিবিসির অবস্থা এখন তাই গতকালের এরপরে বোল ইনিয়ের পরগ মুম অভিজুত ভালবাসি ঢাল সম্পূর্ণ দিকেও শবেস্থানের গুলীচ নাটেশার লগto Her জলটায়ি নদীতে বলেভুত পৌছানো চেঞ্জ শিখা ছলে নিতান্তই

Figure 3.9: Example of Gradient Attack

ing each iteration, the trigger is updated to maximize its adversarial effectiveness, ensuring that it consistently manipulates the model's output regardless of the original input text. The process is repeated for up to 100 iterations per input, or until the attack succeeds. Success is defined as the model producing a prediction that deviates from the correct label in a targeted or untargeted manner, depending on the goal of the attack. One of the most notable features of gradient-based attacks is their universality. The triggers generated are not spe-

cific to a single input but can generalize across a dataset, effectively disrupting the model's performance on a wide range of examples. This makes them particularly dangerous, as a single crafted trigger can compromise the reliability of a model across various contexts. Gradient-based attacks are widely used to evaluate the robustness of NLP models, helping researchers and practitioners understand their vulnerabilities. By exposing the reliance of models on specific patterns or features, these attacks provide valuable insights into the limitations of current architectures and highlight the importance of developing more robust and secure machine learning systems.

3.3 Victim Models

a. Sentiment Analysis: Sentiment analysis is a natural language processing (NLP) technique used to determine the sentiment expressed in a piece of text. It involves analyzing the text to identify whether it conveys a positive, negative, or neutral sentiment. This analysis is valuable for understanding public opinion, customer feedback, social media sentiment, and more.

In sentiment analysis, text data is processed using various techniques, including machine learning algorithms, deep learning models, and lexicon-based methods. Supervised machine learning algorithms, such as support vector machines (SVM), logistic regression, and neural networks, are trained on labeled datasets to classify text into sentiment categories. Deep learning models, such as recurrent neural networks (RNNs), convolutional neural networks (CNNs), and transformer-based architectures like BERT, have also demonstrated high performance in sentiment analysis tasks by capturing complex contextual relationships in text.

Sentiment analysis can be applied to different types of text data, including product reviews, social media posts, news articles, and customer feedback. Businesses use sentiment analysis to monitor brand reputation, assess customer satisfaction, and identify emerging trends. It also has applications in finance for analyzing market sentiment, in healthcare for monitoring patient sentiment, and in politics for gauging public opinion. Here we used the ka05ar/banglabert-sentiment model for bangla sentiment analysis.

¹[ka05ar/banglabert-sentiment](#):

The "ka05ar/banglabert-sentiment" model is a BERT-based model specifically designed for sentiment analysis tasks in the Bangla language. Sentiment analysis involves determining the sentiment expressed in a piece of text, such as whether it is positive, negative, or neutral. This model is pretrained and fine-tuned to accurately classify sentiment in Bangla text.

The model is built on top of the BERT (Bidirectional Encoder Representations from Trans-

formers) architecture, which is known for its ability to capture contextual information from text effectively. BERT models have shown remarkable performance in various NLP tasks due to their bidirectional nature and attention mechanisms. This model is specifically trained on Bangla text data, allowing it to understand the nuances of the Bangla language. It can effectively process and analyze sentiment in Bangla text, taking into account the unique linguistic characteristics of the language. The primary task of this model is sentiment analysis, where it classifies the sentiment of Bangla text into predefined categories such as positive, negative, or neutral. By leveraging the contextual information encoded in the BERT representations, the model can make accurate predictions about the sentiment expressed in Bangla text. The "ka05ar/banglabert-sentiment" model is pretrained on a large corpus of Bangla text data and fine-tuned on sentiment analysis datasets. This process helps the model adapt its representations to the specific task of sentiment analysis, improving its performance on sentiment-related tasks.

The model's performance is evaluated on standard sentiment analysis benchmarks and datasets in the Bangla language. Metrics such as accuracy, precision, recall, and F1 score are used to assess the model's ability to correctly classify sentiment in Bangla text.

b. HateSpeech detection: Hate speech detection is a natural language processing (NLP) task aimed at identifying and categorizing text that contains hateful, offensive, or derogatory language targeted at individuals or groups based on attributes such as race, ethnicity, religion, gender, sexual orientation, disability, or nationality. The detection of hate speech is crucial for maintaining online safety, promoting respectful discourse, and preventing the spread of harmful content.

In hate speech detection, machine learning models are trained to automatically classify text into categories such as hate speech, offensive language, or non-offensive language. These models learn from labeled datasets where each text sample is annotated with its corresponding hate speech label. Supervised learning algorithms, such as support vector machines (SVM), logistic regression, and deep neural networks, are commonly used for hate speech detection tasks.

Features used in hate speech detection models may include lexical features (e.g., specific keywords associated with hate speech), syntactic features (e.g., grammatical patterns), and semantic features (e.g., sentiment analysis). Additionally, contextual information and linguistic cues are important for accurately detecting hate speech, as the meaning of a text can depend heavily on its context.

Hate speech detection models face several challenges, including the diversity and evolution of hate speech language, the presence of sarcasm and irony, and the need to balance freedom of speech with the prevention of harm. As a result, ongoing research is focused on devel-

¹<https://huggingface.co/ka05ar>

oping more robust and accurate hate speech detection methods, including the use of deep learning techniques and cross-lingual approaches to address multilingual contexts. Here we used the Hate-speech-CNERG/bengali-abusive-MuRIL model for bangla HateSpeech detection.

²Hate-speech-CNERG/bengali-abusive-MuRIL:

The "Hate-speech-CNERG/bengali-abusive-MuRIL" model is a specialized model designed for detecting abusive and hate speech content in the Bengali language. Abusive and hate speech detection is a crucial task in natural language processing (NLP) aimed at identifying and flagging content that contains offensive, derogatory, or harmful language. The model is based on the Multilingual Representations for Indian Languages (MuRIL) architecture. MuRIL is a multilingual BERT-based model trained on text data from multiple Indian languages, including Bengali. It is specifically designed to understand and process text in Indian languages effectively.

The "Hate-speech-CNERG/bengali-abusive-MuRIL" model is pretrained and fine-tuned on Bengali text data, enabling it to understand the linguistic nuances and context of the Bengali language. It can effectively analyze text written in Bengali and detect abusive or hate speech content.

The primary task of this model is to detect abusive and hate speech content in Bengali text. It classifies text into categories such as abusive, non-abusive, hate speech, and non-hate speech based on the presence of offensive language, harmful intent, or discriminatory content. The model is pretrained on a large corpus of text data, including general Bengali text as well as labeled datasets containing abusive and hate speech content. It is then fine-tuned on specific abusive and hate speech detection datasets to adapt its representations to the task at hand. The performance of the "Hate-speech-CNERG/bengali-abusive-MuRIL" model is evaluated using standard metrics such as accuracy, precision, recall, and F1 score. These metrics measure the model's ability to correctly classify text into abusive and non-abusive categories and its effectiveness in detecting hate speech content.

c. FakeNews Detection: Fake news detection is the process of identifying and classifying misinformation or false information presented as news. With the rise of social media and online platforms, the dissemination of fake news has become a significant concern due to its potential to mislead the public, influence opinions, and disrupt democratic processes.

Challenges in fake news detection include the rapidly evolving nature of fake news tactics, the prevalence of clickbait headlines and sensationalist content, and the need to balance freedom of speech with the prevention of harm. Ongoing research in this area focuses on developing more sophisticated algorithms, leveraging deep learning techniques, and in-

²<https://huggingface.co/Hate-speech-CNERG/bengali-abusive-MuRIL>

tegrating contextual information to improve the accuracy and effectiveness of fake news detection systems.

fake news detection is essential for combating misinformation, preserving the integrity of the news ecosystem, and promoting informed decision-making among the public. It requires a multidisciplinary approach involving technology, journalism, and public awareness to address the complex challenges posed by fake news. Here we used the armansakif/bengali-fake-news model for bangla FakeNews Detection.

³[armansakif/bengali-fake-news](https://huggingface.co/armansakif/bengali-fake-news):

The "armansakif/bengali-fake-news" model is a specialized model designed for detecting fake news in the Bengali language. Fake news detection is a critical task in natural language processing (NLP) aimed at identifying and flagging misinformation, disinformation, and misleading content. The model is pretrained and fine-tuned on Bengali text data, enabling it to understand the linguistic nuances and context of the Bengali language. It can effectively analyze text written in Bengali and detect patterns indicative of fake news or misinformation. The primary task of this model is to detect fake news content in Bengali text. It classifies text into categories such as fake news and non-fake news based on various features, including the credibility of the source, the accuracy of the information, and the presence of misleading or false claims. The model is pretrained on a large corpus of text data, including general Bengali text as well as labeled datasets containing examples of fake news. It is then fine-tuned on specific fake news detection datasets to adapt its representations to the task at hand.

The performance of the "armansakif/bengali-fake-news" model is evaluated using standard metrics such as accuracy, precision, recall, and F1 score. These metrics measure the model's ability to correctly classify text into fake news and non-fake news categories and its effectiveness in detecting misinformation.

d. Sarcasm Detection: Sarcasm detection is the process of identifying instances of sarcasm in text. Sarcasm is a form of irony characterized by saying something but meaning the opposite, often to convey humor, criticism, or mockery. In sarcasm detection, natural language processing (NLP) techniques are used to analyze linguistic cues, contextual information, and semantic nuances to determine whether a text expresses sarcasm.

Challenges in sarcasm detection include the subtlety and ambiguity of sarcasm, cultural and contextual variations in sarcastic expressions, and the need to distinguish sarcasm from other forms of irony or humor. Ongoing research focuses on developing more sophisticated algorithms, leveraging deep learning techniques, and incorporating multimodal features (e.g., audio and visual cues) to improve sarcasm detection accuracy.

³<https://huggingface.co/armansakif/bengali-fake-news>

Sarcasm detection plays a crucial role in understanding the nuances of human communication, enhancing sentiment analysis systems, and improving the performance of NLP applications in domains such as social media monitoring, customer feedback analysis, and opinion mining. Here we used the raquiba/sarcasm-detection-BanglaSARC model for bangla Sarcasm Detection.

⁴[raquiba/sarcasm-detection-BanglaSARC](https://huggingface.co/raquiba/sarcasm-detection-BanglaSARC):

The "raquiba/sarcasm-detection-BanglaSARC" model is a specialized model designed for detecting sarcasm in the Bengali language. Sarcasm detection is a challenging task in natural language processing (NLP) aimed at identifying instances of sarcasm or irony in text. This model is pretrained and fine-tuned on Bengali text data, enabling it to understand the linguistic nuances and context of the Bengali language. It can effectively analyze text written in Bengali and detect patterns indicative of sarcasm.

The primary objective of this model is to classify text into categories such as sarcastic and non-sarcastic based on various linguistic features, contextual cues, and semantic incongruities. It leverages machine learning techniques to identify instances of sarcasm in Bengali text. The model is pretrained on a large corpus of Bengali text data, which includes both general text as well as labeled datasets containing examples of sarcasm. Subsequently, it undergoes fine-tuning on specific sarcasm detection datasets to adapt its representations and parameters to the nuances of the task at hand. The performance of the "raquiba/sarcasm-detection-BanglaSARC" model is evaluated using standard metrics such as accuracy, precision, recall, and F1 score. These metrics measure the model's ability to correctly classify text into sarcastic and non-sarcastic categories and its effectiveness in detecting sarcasm in Bengali text.

e. ClickBait Detection: Clickbait detection is the process of identifying headlines or content designed to attract attention and encourage users to click on a link, often by using sensationalist or misleading language. Clickbait is commonly used in online media and social media to increase traffic to websites or articles. Detecting clickbait is important for maintaining transparency, trustworthiness, and user experience on online platforms.

Challenges in clickbait detection include the subjective nature of clickbait, cultural and contextual variations in what constitutes clickbait, and the need to balance the detection of clickbait with the preservation of freedom of expression.

Clickbait detection helps users make informed decisions about which content to engage with online, promotes transparency and trust in online media, and contributes to a more meaningful and authentic online experience. Here we used the samanjoy2/banglaclickbert_base model for bangla ClickBait Detection.

⁴<https://huggingface.co/raquiba/sarcasm-detection-BanglaSARC>

⁵[samanjoy2/banglaclickbert_base](https://huggingface.co/samanjoy2/banglaclickbert_base):

The "samanjoy2/banglaclickbert_base" model is a BERT-based model specifically designed for clickbait detection in the Bengali language. Clickbait detection is a crucial task in natural language processing (NLP) aimed at identifying headlines or titles that are sensationalized or misleading with the intention of enticing readers to click on them. The model is built upon the BERT (Bidirectional Encoder Representations from Transformers) architecture, which is known for its effectiveness in capturing contextual information from text effectively. BERT models have shown remarkable performance in various NLP tasks due to their bidirectional nature and attention mechanisms. This model is pretrained and fine-tuned on Bengali text data, enabling it to understand the linguistic nuances and context of the Bengali language. It can effectively analyze text written in Bengali and detect patterns indicative of clickbait content.

The primary task of this model is to classify text into categories such as clickbait and non-clickbait based on various linguistic features, structural elements, and content characteristics. It leverages machine learning techniques to discern between genuine headlines and sensationalized or misleading titles. The model is pretrained on a large corpus of Bengali text data, which includes both general text as well as labeled datasets containing examples of clickbait headlines. Subsequently, it undergoes fine-tuning on specific clickbait detection datasets to adapt its representations and parameters to the intricacies of the task at hand. The performance of the "samanjoy2/banglaclickbert_base" model is evaluated using standard metrics such as accuracy, precision, recall, and F1 score. These metrics measure the model's ability to correctly classify text into clickbait and non-clickbait categories and its effectiveness in detecting sensationalized or misleading headlines in Bengali text.

⁵https://huggingface.co/samanjoy2/banglaclickbert_base

Chapter 4

Dataset Description

Chapter 4 is dedicated to examining the statistical data in detail. The "Data Statistics" segment outlines details about the dataset, encompassing its source, methodology of collection, and distinctive features. Additionally, it furnishes statistical details such as the size of the dataset, distribution of classes, and the quantity of data within each class.

4.1 Data Statistics

We have used five datasets here for five different classification models. SentNoB dataset for sentiment analysis task and rest of the four datasets for four different classification models that are fake news detection, sarcasm detection, clickbait, hate speech detection that visually represent in the table. 4.1. Utilized dataset for detailed descriptions of the models below :

Dataset Name	No of Samples	Label(s)	Sample(s)
¹ SentNoB	1,567	Negative(label-0)	361
		Positive(label-1)	654
		Neutral(label-2)	571
² Fake News Dataset	14,537	Real(label-1)	10,000
		Fake(label-0)	4,537
³ BanglaSARC	5,111	label-1	1952
		label-0	3,159
⁴ BanglaBait	3,013	Clickbait(label-1)	1,061
		Non-clickbait(label-0)	1,952
⁵ BD-SHS	5029	HateSpeech(label-1)	2,416
		Not-HateSpeech(label-0)	2,613

Table 4.1: Dataset summary

a. SentNoB : For the sentiment analysis model, we utilized the SentNoB dataset, which comprises three labels: positive assigned to label-1, negative designated as label-0, and neutral categorized under label-2. The SentNoB dataset consists of a total of 15,728 samples, distributed across three sets: training, validation, and test. The training set comprises 12,575 samples, the validation set contains 1,567 samples, and the test set includes 1,586 samples. We have considered the test dataset for attacking the model. The test set consists of 361 samples for label-0, 654 samples for label-1, and the remaining 571 samples for label-2.

b. Bangla And English Fake News Detection DataSet : For our fake news detection model, we employed the "Bangla And English Fake News Detection DataSet" sourced from Kaggle. Specifically, we utilized the `final_bn_data.csv` file for our analysis. This dataset encompasses various news categories, with National news comprising 37%, International news 13%, and the remaining percentage allocated to Other news. The dataset operates on two labels: real for label-1 and fake for label-0. The dataset comprises 14,537 samples, with 4,537 samples classified as label-0, representing fake news, and 10,000 samples classified as label-1 representing real news.

c. BanglaSARC : For sarcasm detection tasks, we utilized the "BanglaSarc" datasets obtained from Kaggle. The dataset operates across two levels: label-1 and label-0. The BanglaSarc datasets contain a total of 5,111 samples, with 3,159 samples classified as label-0 and the remaining samples categorized as label-1.

d. BanglaBait : For our Clickbait detection tasks, we employed the "BanglaBait" dataset sourced from Kaggle. This dataset is segmented into train, test, dev, and unlabelled_data sets. For our analysis, we utilized the test.csv set. The test set comprises 3,013 samples, with 1,952 samples classified as label-0, and the remaining 1,061 samples categorized as label-1. Label-1 denotes Clickbait, while label-0 represents non-clickbait.

e. BD-SHS : For our Hatespeech detection task, we utilized the BD-SHS dataset obtained from Kaggle. BD-SHS is a benchmark dataset designed for detecting online Bangla hate speech across various social contexts. This dataset is divided into train, test, and validation sets, and we specifically chose the test set for model evaluation. The test set comprised 5,029 samples, with 2,613 samples categorized as label-0 and the remaining 2,416 samples categorized as label-1. Label-1 is classified as hate speech, while label-0 is classified as non-hate speech.

¹<https://www.kaggle.com/datasets/cryptexcode/sentnob-sentiment-analysis-in-nois>

²<https://www.kaggle.com/datasets/sadikaljarif/bangla-and-english-fake-news-data>
final_bn_data.csv

³<https://www.kaggle.com/datasets/sakibapon/banglasarc?select=SarcasDetection.csv>

⁴<https://www.kaggle.com/datasets/motaharmahtab/banglabait-bangla-clickbait-data>

⁵<https://www.kaggle.com/datasets/naurosromim/bdshs>

Chapter 5

Methodology

5.1 Problem Formulation

Initially, we established the framework for textual adversarial attacks specifically aimed at text classification. We are given a pre-trained Bangla text classification model $f : x \rightarrow y$ and an input text X containing a set of samples $X = \{x_1, x_2, x_3, \dots, x_n\}$ and their corresponding original prediction level $Y = \{y_1, y_2, y_3, \dots, y_n\}$.

When provided with an original input x , adversarial examples are crafted to induce errors in a model's predictions. In the soft-label black-box scenario, we presume that the attacker can solely request output probabilities from the classifier for a given input, without access to the model's parameters, gradients, or training data (Garg et al., 2020 [15]). For a set of input $X = \{x_1, x_2, x_3, \dots, x_n\}, Y$ we aim to generate an adversarial example $X' = \{x'_1, x'_2, x'_3, \dots, x'_n\}, Y'$, whose original level is still Y but the adversarial example result will be shifted to Y' , $f(X') \neq f(X)$.

To ensure that the perturbation is small enough to be invisible to humans, the adversarial examples must meet criteria related to vocabulary, grammar, and meaning. The lexical rule says don't swap the right word with a wrong one that's often misspelled because spell check will catch it easily before the classifier sees it. Also, the changed examples need to stay grammatically correct. Lastly, any modifications made to the original samples should not substantially alter their semantics, ensuring compliance with semantic constraints.

To meet the above constraints we paraphrase and mask the input text to generate adversarial examples. Text attack models are grouped based on how much they change the original input: whole sentences, individual words, or even single characters. Sentence level attack models create adversarial examples mainly doing things like adding distracting sentences (Jia and Liang, 2017 [19]), paraphrasing (Iyyer et al. [20], 2018; Ribeiro et al., 2018 [23]) or text generation by encoder-decoder (Zhao et al., 2018 [18]). Word-level attack model

basically concentrate with word substitution, namely replacing some words in the original input text with semantically identical or similar words such as synonyms (Jin et al., 2020 [8]; Ren et al., 2019 [11]; Alzantot et al., 2018 [16]). Masking is a way to do this job correctly toward robustness (Zeng et al., 2023 [9]). In character-level attack models, changes are made to individual letters, such as swapping, replacing, removing, adding, or repeating them (Eger et al., 2019 [22]; Ebrahimi et al., 2018 [21]; Belinkov and Bisk, 2018 [24]).

5.2 Paraphrasing

For each given input we paraphrase the input text to generate adversarial examples. Firstly we tokenized input using Bangla sentence tokenizer which is shown on Algorithm-1 in line 2.

Algorithm 1: Generate perturbed text using paraphrase

Input: Legitimate document X and its original truth label Y , classifier $f(\cdot)$

Output: Adversarial document X_{adv}

```

1 Initialization:  $X_{adv} \leftarrow X$ ;
2 for  $x_i$  in  $X$  do
3   Sentence_list  $\leftarrow$  Tokenize  $x_i$  using sentence tokenizer;
4   text  $\leftarrow$  "";
5   for sentence in Sentence_list do
6     Paraphrase_text  $\leftarrow$  GenParaphrases(sentence);
7     text  $\leftarrow$  Concatenate(text, Paraphrase_text);
8    $X_{adv} \leftarrow$  text;
9   if  $f(X_{adv}) \neq y$  then
10     Attack Successful;
11   else
12     Attack unsuccessful;
```

Motivation is to tokenize the input is that we have used “csebuetnlp/banglat5_banglaparaphrase” model which is briefly describe in background section. We have fixed the max_length of the model to 512 . So if the input size is too long then the whole input doesn’t paraphrase properly. In Fact it paraphrases the first 512 length of our input.

$$X_i = w_1 w_2 w_3 w_4 \dots w_n$$

Here w_i is the i th sentence token. After tokenize the given input we forward every token on to GenParaphrases model which is “csebuetnlp/banglat5_banglaparaphrase”. GenParaphrase model returns me a paraphrase text for forwarded sentence token. Then we concat

all the paraphrase tokens and store it on a variable that is represented on algorithm-1 in line 7.

$$X'_i = w'_1 w'_2 w'_3 \dots w'_n$$

Here w'_i is the paraphrased output of w_i token. And X'_i is the paraphrase result of X_i with concatenation of paraphrased text in line 7 (Algorithm-1) of every sentence token that is $w \in X$.

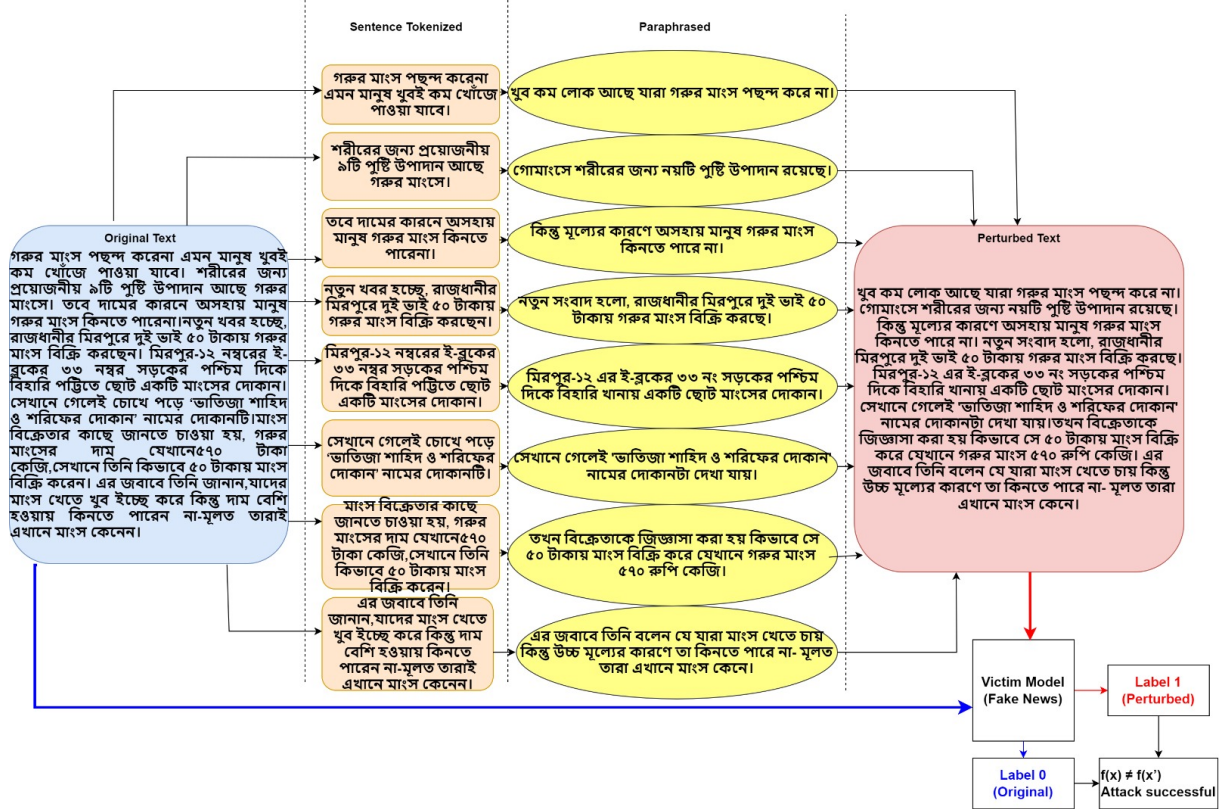


Figure 5.1: Visual Representation of Paraphrase Algorithm

After generating paraphrase examples we then send it to targeted victim models FakeNews-Detection, Sarcasm Detection, HateSpeech detection, ClickBait detection and Sentiment detection. After getting a probability label from the victim model we will check whether it is matched with the original label or not. If it is mismatched with the original label then our attack is successful otherwise not.

5.3 Random Masking

At a random position of the input sequence we will apply masking. Using masking three perturbation techniques already discussed (Li et al., 2020 [5]), from here we used Replace action for our task. For masking, we'll start by splitting the input text into sentences using the sentence tokenizer.

Algorithm 2: Generate perturbed text using masking

Input: Legitimate document X and its original truth label Y , classifier $f(\cdot)$

Output: Adversarial document X_{adv}

```

1 for  $x_i$  in  $X$  do
2   Sentence_list  $\leftarrow$  Tokenize  $x_i$  using sentence tokenizer;
3   text  $\leftarrow$  " ";
4   for sentence in Sentence_list do
5     pos  $\leftarrow$  Select a random word position in sentence;
6     masked_text  $\leftarrow$  sentence[0, pos-1] [MASK] sentence[pos+1, n];
7     // n is the number of tokens
8     T  $\leftarrow$  Predict top-k tokens for [MASK] in masked_text;
9     fill_masked  $\leftarrow$  Replace [MASK] with T[0];
10    text  $\leftarrow$  Concatenate(text, fill_masked);
11   $X_{adv} \leftarrow$  text;
12  if  $f(X_{adv}) \neq y$  then
13    Attack Successful;
14  else
15    Attack Not Successful;
```

We will select a random position using the rand function between the length of the sentence. It first replaces x_{pos} with the [MASK] token and then selects a token t from a candidate set T to fill in:

$$X' = x_{[0,pos-1]}[MASK]x_{[pos+1,length]}$$

$$\text{fill_mask}(X', pos) = x_{[0,pos-1]}T[0]x_{[pos+1,length]}.$$

Here to produce an adversarial example we fill masked with t which should be fit into the unmasked context. We select t with high probability and $X'_t(\text{fill_masked}(X', pos))$ should be similar to X (Li et al., 2020 [5]). Then we concat all the fill_masked sentence token and store it on a variable. After creating paraphrased examples, we send them to different models like FakeNewsDetection, Sarcasm Detection, HateSpeech Detection, ClickBait Detection, and Sentiment Detection. Then, we compare the probability levels obtained from these models with the original ones. If they don't match, our attack is successful; otherwise, it's not.

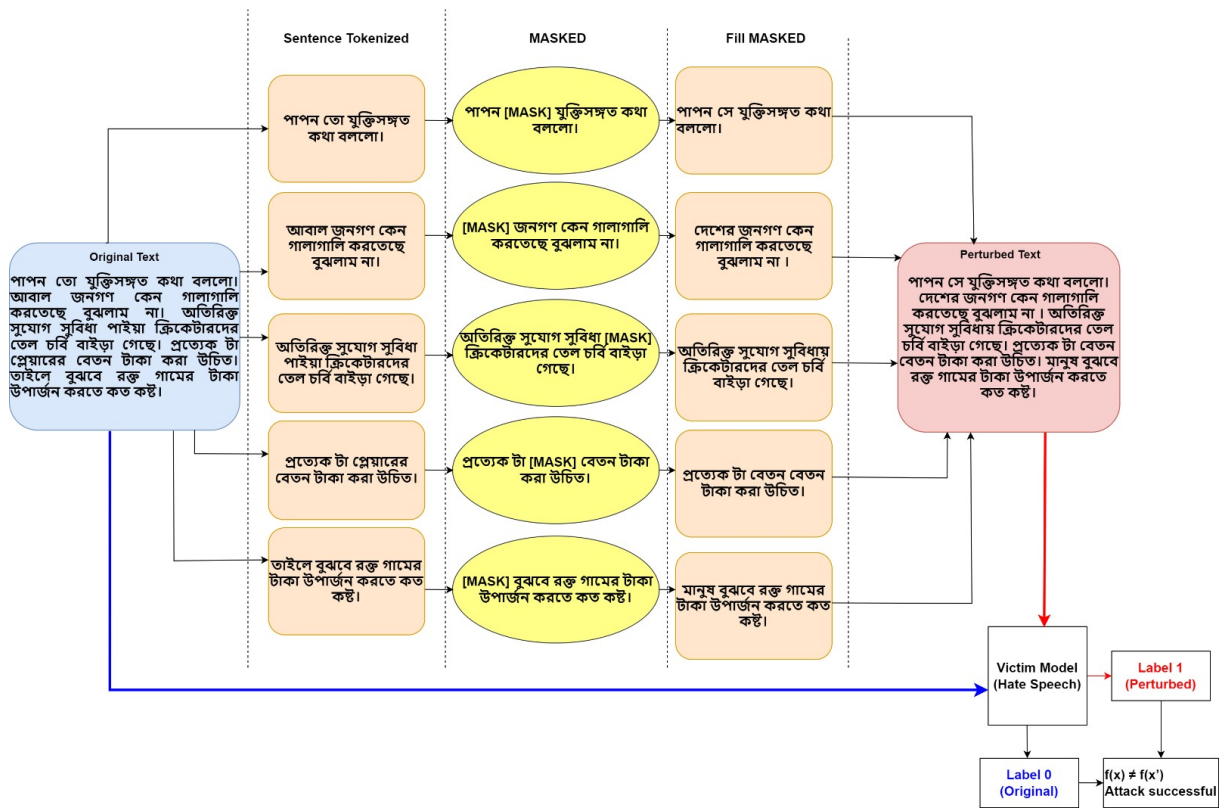


Figure 5.2: Visual Representation of Random Masking Algorithm

5.4 Important Word Masking

Important word masking refers to a technique used to perturb or obscure key parts of a text while maintaining its overall structure and meaning. The goal is to alter important words without changing the general understanding of the content, making it useful for evaluating model robustness or generating adversarial examples (Malik et al., 2020 [39]). This method helps identify which words are crucial for a model's predictions while still preserving the core context of the input.

Algorithm 3: Generate perturbed text using important word masking

Input: Legitimate document X and its original truth label Y , classifier $f(\cdot)$

Output: Adversarial document X_{adv}

```

1 for  $x_i$  in  $X$  do
2   Original_pre  $\leftarrow f(x_i)$ ;
3   Word_list  $\leftarrow$  Tokenize  $x_i$  using word tokenizer;
4   text  $\leftarrow$  empty string;
5   val[]  $\leftarrow \emptyset$ ;
6   for word in Word_list do
7     next_text  $\leftarrow$  Word_list without word;
8     dis  $\leftarrow$  Original_pre  $- f(\text{next\_text})$ ;
9     val.append([dis, pos]);
10  sort_list  $\leftarrow$  sort val by dis and select up to  $k$  max values;
11  //Set  $k = \text{length}(\text{Word\_list})/10, k > 10??k = 10$ 
12  for  $i$  in sort_list do
13    pos  $\leftarrow i$ ;
14    masked_text  $\leftarrow$  sentence[0 : pos - 1] + [MASK] + sentence[pos + 1 : n];
15    //  $n$  is the number of tokens
16     $T \leftarrow$  Predict top- $k$  tokens for [MASK] in masked_text;
17    fill_masked  $\leftarrow$  Replace [MASK] with  $T[0]$ ;
18    text  $\leftarrow$  fill_masked;
19   $X_{adv} \leftarrow$  text;
20  if  $f(X_{adv}) = Y$  then
21    return Attack Successful;
22  else
23    return Attack not Successful;
```

Here, The input text is tokenized into words using a word tokenizer. To identify important words, one token is removed at a time from the tokenized sentence, and the modified sentence is sent to the model. The difference between the original prediction and the prediction without the token is measured, and this difference is tracked along with the token's index. The token with the largest difference from the original prediction is identified as the most important word. Its index is then sent to the masking model to obscure it. The model's token length is fixed at 512 tokens, and important words are identified in groups of 10 tokens.

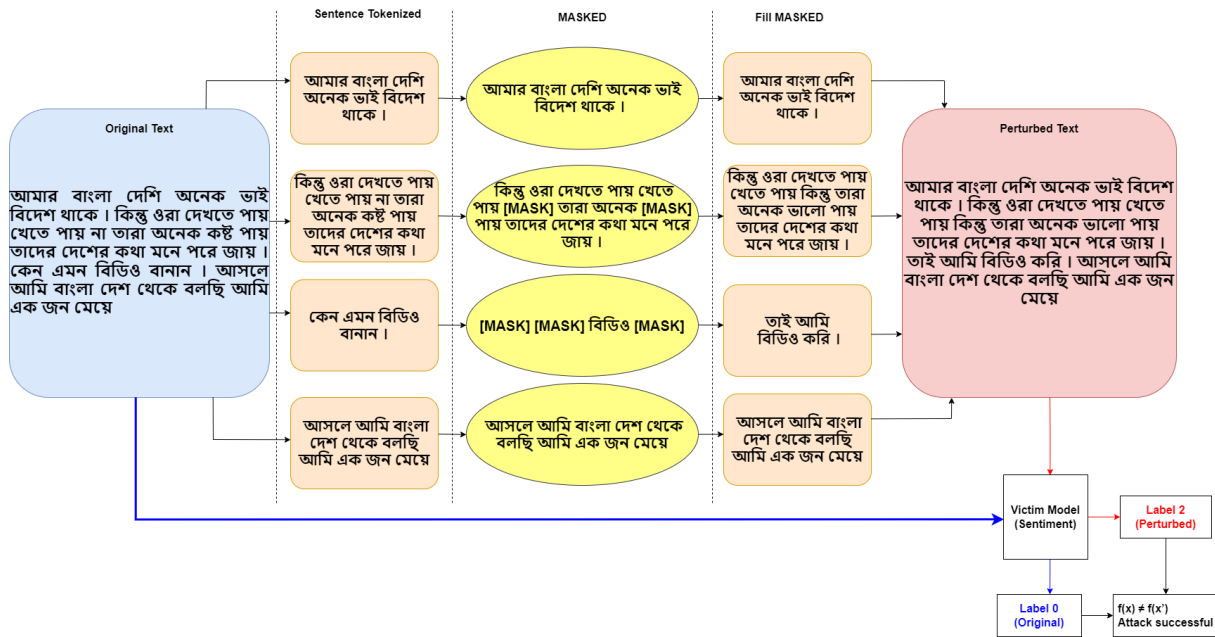


Figure 5.3: Visual Representation of Important Word Masking Algorithm

A maximum of 10 tokens can be masked per input. For Fake News and Clickbait Detection datasets, which exceed 512 tokens, the average token length across the full dataset is considered to manage input size efficiently. Other datasets remain below 512 tokens Algorithm-3.

5.5 Homoglyph Attack

A homoglyph attack is a specialized adversarial attack in natural language processing (NLP) where visually similar characters, known as homoglyphs, are used to replace certain characters in text (Creo et al., 2024 [38]). The purpose of this attack is to subtly modify text in a way that retains its readability for humans but introduces ambiguity or misleads machine learning models, particularly text classifiers.

Homoglyph attacks exploit the fact that many characters across different languages or scripts look almost identical but are treated as distinct by computational systems. For instance, in Bengali, characters such as shown in Figure-3.4. These substitutions appear visually similar to human readers, ensuring the semantic and structural coherence of the text, but they can disrupt the functioning of NLP models that rely heavily on token-based or character-based representations.

Algorithm 4: Generate perturbed text using homoglyph

Input: Legitimate document X and its original truth label Y , classifier $f(\cdot)$

Output: Adversarial document X_{adv}

```

1 for  $x_i$  in  $X$  do
2   Word_list  $\leftarrow$  Tokenize  $x_i$  using word tokenizer;
3   text  $\leftarrow$  "";
4   for word in Word_list do
5     Char_list  $\leftarrow$  Split it into character;
6     Word  $\leftarrow$  "";
7     for char in Char_list do
8       Char_list[pos]  $\leftarrow$  Replace[র] with [ৱ], [স] with [ম], [ম]
          with [ম], [ব] with [ৰ], [এ] with [ঐ];
9     Word  $\leftarrow$  Concatenate(Char_list);
10  text  $\leftarrow$  Concatenate(Word_list);
11  $X_{adv} = \text{text}$ ;
12 if  $f(X_{adv}) \neq y$  then
13   Attack Successful;
14 else
15   Attack Not Successful;
```

In the context of a text classification task, such as fake news detection, a homoglyph attack can be used to evaluate the robustness of the model by replacing characters in the input text and observing changes in the model's predictions. For example, by applying these substitutions to the first 100 or 200 tokens of a dataset, researchers can determine how

sensitive the model is to small perturbations in its input. Results from experiments often show that extending the attack to a larger portion of the text, such as the first 200 tokens, tends to yield more significant outcomes, as it increases the likelihood of disrupting the model's understanding of the input.

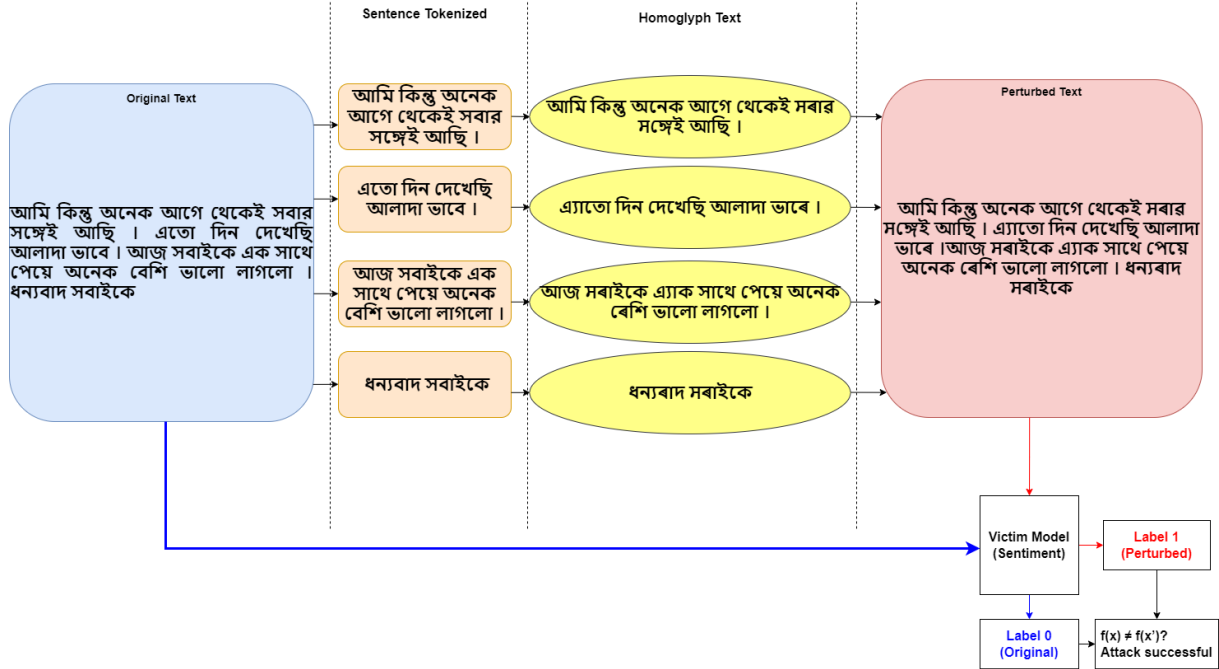


Figure 5.4: Visual Representation of Homoglyph Attack Algorithm

Homoglyph attacks are particularly useful for stress-testing models in real-world scenarios, as they mimic potential adversarial behaviors in user-generated content, such as social media posts or comments. By analyzing the effects of homoglyph attacks, researchers can identify vulnerabilities in NLP systems and develop strategies to enhance their robustness, such as improving character normalization processes or incorporating adversarial training techniques.

5.6 Gradient Attack

Gradient-based attacks leveraging universal adversarial triggers are an effective method for exploiting vulnerabilities in natural language processing (NLP) models. These triggers are short, fixed sequences of words or phrases that, when appended to different input texts, consistently lead the model to make incorrect predictions (Guo, Chuan, et al., 2021 [40]). Unlike input-specific adversarial attacks, universal triggers are designed to generalize across multiple inputs, making them versatile and potent. The trigger selection process often begins with a sequence derived from prior research or empirical evidence, which is then refined using gradient-based optimization. Gradients are computed to identify how the trigger influences the model's predictions, and adjustments are made to maximize its disruptive effect.

Algorithm 5: Generate perturbed text using gradient attack

Input: Legitimate document X and its original truth label Y , classifier $f(\cdot)$

Output: Adversarial document X_{adv}

```

1 for  $x_i$  in  $X$  do
2    $text \leftarrow ""$ ;
3   for  $i$  in  $range(100)$  do
4      $text = x_i + \text{add trigger}$  ;
5     //Generate trigger using Greedy Coordinate Gradient-based Search
6     if  $f(X_{adv}) \neq y$  then
7       Attack Successful;
8        $X_{adv} = text$ ;
9       break;
10  if  $X_{adv} = NULL$  then
11    Attack unsuccessful;

```

To evaluate the attack's success, the trigger is appended to an input text and sent to the model. If the model's prediction remains unaffected, the trigger is iteratively updated using feedback from the model, such as gradient signals, to enhance its effectiveness. This process is repeated for up to 100 iterations per input, balancing computational efficiency with attack success. A successful attack occurs when the trigger consistently causes incorrect predictions across various inputs. Such attacks not only demonstrate the susceptibility of NLP models to adversarial manipulation but also raise critical concerns about their robustness in real-world applications.

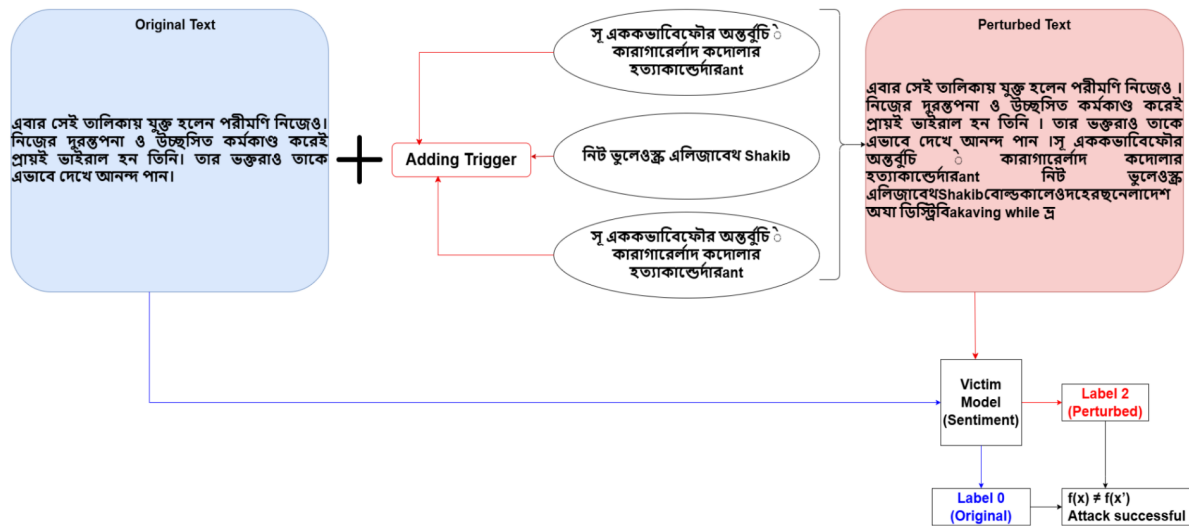


Figure 5.5: Visual Representation of Gradient Attack Algorithm

Chapter 6

Evaluation

In this study, we conducted experiments to assess the effectiveness of our adversarial attack on five pre-trained models using five distinct attacking methods: paraphrasing, random masking, homoglyph, important word masking, gradient attack, specifically tailored for Bangla text. Throughout the experiment, we employed five datasets comprising diverse Bangla text samples, as detailed in the preceding section.

6.1 Experimental Result

To determine the attack success rate of an attack method, it is essential to first establish the original accuracy of the victim model. Then, after performing the attack, we calculate the accuracy of the targeted victim model. By comparing the after-attack accuracy with the original accuracy, we can measure the change rate, which indicates the effectiveness of the attack in manipulating the model's predictions. This change rate serves as a metric for assessing the success of the attack method in deceiving the victim model.

Datasets	Gradient Attack		Important Word Masking		
	Original accuracy	Attack Success Rate	Before Attack Changed Rate	After Attack Changed Rate	Attack Success Rate
Clickbait	65.72	35.28	36.09	35.42	11.04
Fakenews	74.55	91.9	25.45	53.09	57.61
Hatespeech	87.89	96.56	12.11	48.04	40.11
Sarcasm	98.88	98.81	1.12	38.22	38.56
Sentiment	16.02	99.56	83.98	77.3	34.74

Table 6.1: Table showing various attack rates for different techniques

Datasets	Homoglyph			Random Word Masking			Paraphrase		
	Before Attack Changed Rate	After Attack Changed Rate	Attack Success Rate	Before Attack Changed Rate	After Attack Changed Rate	Attack Success Rate	Before Attack Changed Rate	After Attack Changed Rate	Attack Success Rate
Clickbait	34.28	60.30	76.92	35.73	37.68	8.77	62.88	64.18	8
Fakenews	25.45(100w) 25.45(200w)	28.4(100w) 34.66(200w)	9.95(100w) 23.61(200w)	25.46	52.27	55.91	25.45	68.35	89.19
Hatespeech	12.11	15.89	6.24	12.09	16.35	5.53	12.11	20.26	11.14
Sarcasm	1.13	31.73	31.96	1.13	2.56	2.13	1.12	19.33	18.92
Sentiment	83.97	80.13	19.05	83.73	81.72	9.02	83.98	79.63	21.56

Table 6.2: Table showing various attack rates for different techniques

6.2 Experimental Result Analysis

As shown in Table 6.2, the Homoglyph attack demonstrated the highest success rate of 76.92%, with a substantial change in predictions from 34.28% to 60.30% for the Clickbait dataset,. In contrast, the Random Word Masking and Paraphrase attacks were less effective, achieving success rates of only 8.77% and 8%, respectively. The Fakenews dataset showed similar patterns, with Paraphrase attacks being the most effective, reaching an attack success rate of 89.19% after altering predictions significantly. Homoglyph attacks in this dataset were less successful, particularly when using 200 words, resulting in a 23.61% success rate. The Random Word Masking method performed moderately, with a 55.91% success rate. For the Hatespeech dataset, all techniques were less effective, with success rates ranging from 5.53% to 11.14%. The Paraphrase attack showed slight superiority, especially in models detecting hate speech. The Sarcasm dataset was more sensitive to the attacks, with the Homoglyph attack achieving a success rate of 31.96%, while Paraphrase and Random Word Masking attacks were notably less effective, with success rates of 18.92% and 2.13%, respectively. Finally, in the Sentiment dataset, the Paraphrase attack had the highest success rate at 21.56%, whereas the Homoglyph and Random Word Masking attacks had lower success rates, 19.05% and 9.02%, respectively.

In Table 6.1, focusing on Gradient Attack and Important Word Masking, the Gradient Attack consistently outperformed Important Word Masking across all datasets. For example, in the Clickbait dataset, Gradient Attack achieved a success rate of 35.28%, while Important Word Masking had a much lower success rate of 11.04%. The Fakenews dataset saw a significant improvement with Gradient Attack, which achieved a 91.9% success rate, while Important Word Masking had a success rate of 57.61%. Similarly, in the Hatespeech dataset, Gradient Attack showed a remarkable success rate of 96.56%, whereas Important Word Masking was only 40.11%. In Sarcasm, Gradient Attack again outperformed Important Word Masking with a near-perfect success rate of 98.81%, compared to 38.56%. In the Sentiment dataset, Gradient Attack achieved an almost flawless success rate of 99.56%, while Important Word Masking showed a lower success rate of 34.74%.

In conclusion, Gradient Attack emerged as the most powerful technique across all datasets, achieving the highest success rates in most cases, particularly for datasets like Sarcasm

and Sentiment. Paraphrase attacks were most effective in the Fakenews and Sentiment datasets, whereas Homoglyph attacks worked best for Clickbait and Sarcasm. Random Word Masking and Important Word Masking were less effective overall, though Important Word Masking showed some promise in specific cases. This analysis highlights the strengths and weaknesses of each attack technique, with gradient-based methods proving most reliable for crafting adversarial examples across diverse NLP tasks.

Chapter 7

Discussion

In this paper we have presented adversarial attacks on Bengali pre-trained language models. We utilize paraphrasing, random masking, important word masking, homoglyph and gradient adversarial attack methods for perturbation. We saw much research on adversarial attacks on different language models like English, Hindi and others but we did not find any research on adversarial attacks on Bengali language. If there is any work on the Bengali language it is very negligible. We choose five pre-trained banglabert models (e.g. sentiment analysis, fake news detection, hate speech detection, click bait, sarcasm detection) for this attack. We only predict the attack success rate for before attack and after attack by analyzing the model's prediction. After analysis we are very hopeful to see the prediction result. In future, we will try to perform more attack methods on these models and our vision is to target more bengali models for this attack. Gradually we will try to cover every bengali model as possible.

Chapter 8

Conclusion

Overall, our study focused on adversarial attacks against state-of-the-art sentiment analysis, Fake News Detection, Hate Speech Detection, Sarcasm Detection, and Clickbait Detection models on Bengali text. Our experimental results reveal the efficacy and efficiency of Paraphrasing and random Masking attacks on Bengali pre-trained models.

The findings demonstrate that Paraphrasing, Random masking, Important Word masking, Homoglyph and Gradient attacks successfully manipulate the predictions of the targeted models across all evaluated tasks. Specifically, we observed significant decreases in accuracy for all models post-attack, indicating the vulnerability of these models to adversarial attacks. The success rates of the attacks were notably high, underscoring the effectiveness of these methods in deceiving the models. Our study highlights the importance of considering adversarial robustness in the development and deployment of NLP models, particularly in applications involving sensitive tasks such as sentiment analysis, fake news detection, hate speech detection, sarcasm detection, and clickbait detection.

References

- [1] https://en.wikipedia.org/wiki/List_of_languages_by_total_number_of_speakers
- [2] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. 2014. Explaining and harnessing adversarial examples. arXiv preprint arXiv:1412.6572.
- [3] Alexey Kurakin, Ian Goodfellow, and Samy Bengio. 2016. Adversarial examples in the physical world. arXiv preprint arXiv:1607.02533.
- [4] Anirban Chakraborty, Manaar Alam, Vishal Dey, Anupam Chattopadhyay, and Debdeep Mukhopadhyay. 2018. Adversarial attacks and defences: A survey. arXiv preprint arXiv:1810.00069.
- [5] Li, Dianqi, et al. "Contextualized perturbation for textual adversarial attack." arXiv preprint arXiv:2009.07502 (2020).
- [6] Wei Emma Zhang, Quan Z Sheng, Ahoud Alhazmi, and Chenliang Li. 2020. Adversarial attacks on deep-learning models in natural language processing: A survey. ACM Transactions on Intelligent Systems and Technology (TIST), 11(3):1–41
- [7] Shahariar, G. & Shawon, Md & Nibir, Anik & Miandad, Md & Mandal, Nibir Chandra. (2022). Automatic back transliteration of Romanized Bengali (Banglish) to Bengali. Iran Journal of Computer Science. 6. 1-12. 10.1007/s42044-022-00122-9.
- [8] Di Jin, Zhijing Jin, Joey Tianyi Zhou, and Peter Szolovits. 2019. Is bert really robust? natural language attack on text classification and entailment. ArXiv, abs/1907.11932.
- [9] Jiehang Zeng, Jianhan Xu, Xiaoqing Zheng, and Xuanjing Huang. 2023. Certified Robustness to Text Adversarial Attacks by Randomized [MASK]. Computational Linguistics, 49(2):395–427.
- [10] Linyang Li, Ruotian Ma, Qipeng Guo, Xiangyang Xue, and Xipeng Qiu. 2020. BERT-ATTACK: Adversarial Attack Against BERT Using BERT. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pages 6193–6202, Online. Association for Computational Linguistics.

- [11] Shuhuai Ren, Yihe Deng, Kun He, and Wanxiang Che. 2019. Generating natural language adversarial examples through probability weighted word saliency. In *Proc. of ACL*.
- [12] Behjati, Melika & Moosavi-Dezfooli, Seyed-Mohsen & Soleymani, Mahdieh & Frossard, Pascal. (2019). Universal Adversarial Attacks on Text Classifiers. 7345-7349. 10.1109/ICASSP2019.8682430.
- [13] Zhu, Kaijie & Wang, Jindong & Zhou, Jiaheng & Wang, Zichen & Chen, Hao & Wang, Yidong & Yang, Linyi & Ye, Wei & Gong, Neil & Zhang, Yue & Xie, Xing. (2023). Prompt-Bench: Towards Evaluating the Robustness of Large Language Models on Adversarial Prompts.
- [14] Eric Wallace, Shi Feng, Nikhil Kandpal, Matt Gardner, and Sameer Singh. 2019. Universal Adversarial Triggers for Attacking and Analyzing NLP. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2153–2162, Hong Kong, China. Association for Computational Linguistics.
- [15] Siddhant Garg and Goutham Ramakrishnan. 2020. BAE: BERT-based Adversarial Examples for Text Classification. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6174–6181, Online. Association for Computational Linguistics.
- [16] Moustafa Alzantot, Yash Sharma, Ahmed Elgohary, Bo-Jhang Ho, Mani Srivastava, and Kai-Wei Chang. 2018. Generating Natural Language Adversarial Examples. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2890–2896, Brussels, Belgium. Association for Computational Linguistics.
- [17] Li, Jinfeng & Ji, Shouling & Du, Tianyu & Li, Bo & Wang, Ting. (2018). TextBugger: Generating Adversarial Text Against Real-world Applications. 10.14722/ndss.2019.23138.
- [18] Zhengli Zhao, Dheeru Dua, and Sameer Singh. 2018. Generating natural adversarial examples. In *Proceedings of ICLR*.
- [19] Jia, Robin, and Percy Liang. "Adversarial examples for evaluating reading comprehension systems." *arXiv preprint arXiv:1707.07328* (2017).
- [20] Iyyer, Mohit, et al. "Adversarial example generation with syntactically controlled paraphrase networks." *arXiv preprint arXiv:1804.06059* (2018).

- [21] Javid Ebrahimi, Anyi Rao, Daniel Lowd, and Deijing Dou. 2018. HotFlip: White-Box Adversarial Examples for NLP. In Annual Meeting of the Association for Computational Linguistics (ACL).
- [22] Steffen Eger, Gözde Gürel, Şahin, Andreas Ruckl, Ji-Ůng Lee, Claudia Schulz, Mohsen Mesgar, Krishnkant Swarnkar, Edwin Simpson, and Iryna Gurevych. 2019. Text processing like humans do: Visually attacking and shielding NLP systems. In Proceedings of NAACL-HLT.
- [23] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2018. Semantically equivalent adversarial rules for debugging NLP models. In Proceedings of ACL.
- [24] Yonatan Belinkov and Yonatan Bisk. 2018. Synthetic and natural noise both break neural machine translation. In Proceedings of ICLR.
- [25] Vaswani, Ashish, et al. "Attention is all you need." *Advances in neural information processing systems* 30 (2017).
- [26] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. 2014. Intriguing properties of neural networks. In Proceedings of ICLR.
- [27] Anh Mai Nguyen, Jason Yosinski, and Jeff Clune. 2015. Deep neural networks are easily fooled: High confidence predictions for unrecognizable images. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2015, Boston, MA, USA, June 7-12, 2015*, pages 427–436.
- [28] Florian Tramer, Alexey Kurakin, Nicolas Papernot, Ian Goodfellow, Dan Boneh, and Patrick McDaniel. 2018. Ensemble adversarial training: Attacks and defenses. In *International Conference on Learning Representations*.
- [29] Eric Wong and J. Zico Kolter. 2018. Provable defenses against adversarial examples via the convex outer adversarial polytope. In *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmassan, Stockholm, Sweden, July 10-15, 2018*, pages 5283–5292.
- [30] Chuanbiao Song, Kun He, Liwei Wang, and John E Hopcroft. 2019. Improving the generalization of adversarial training with domain adaptation. In *The Seventh International Conference on Learning Representations, New Orleans, Louisiana*.
- [31] Yinpeng Dong, Fangzhou Liao, Tianyu Pang, Hang Su, Jun Zhu, Xiaolin Hu, and Jianguo Li. 2018. Boosting adversarial attacks with momentum. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9185–9193.

- [32] Mamta and Asif Ekbal. 2022. Adversarial Sample Generation for Aspect based Sentiment Classification. In Findings of the Association for Computational Linguistics: ACL-IJCNLP 2022, pages 478–492, Online only. Association for Computational Linguistics.
- [33] Das, Sourya Basak, Ayan Mandal, Soumil Das, Dipankar. (2021). AdvCodeMix: Adversarial Attack on Code-Mixed Data. 10.48550/arXiv.2111.00350.
- [34] X. Li, H. Luo and Y. Sun, "A Lightweight Chinese Multimodal Textual Defense Method based on Contrastive-Adversarial Training," 2024 International Joint Conference on Neural Networks (IJCNN), Yokohama, Japan, 2024, pp. 1-10, doi: 10.1109/IJCNN60899.2024.10649990. keywords: Training;Costs;Accuracy;Text categorization;Semantics;Neural networks;Feature extraction;Chinese Text Classification;Adversarial Samples;Model Robustness;Textual Defense,
- [35] Morris, John X. et al. "TextAttack: A Framework for Adversarial Attacks, Data Augmentation, and Adversarial Training in NLP" Conference on Empirical Methods in Natural Language Processing (2020).
- [36] Guoyang Zeng, Fanchao Qi, Qianrui Zhou, Tingji Zhang, Zixian Ma, Bairu Hou, Yuan Zang, Zhiyuan Liu, and Maosong Sun. 2021. OpenAttack: An Open-source Textual Adversarial Attack Toolkit. In Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations, pages 363–371, Online. Association for Computational Linguistics.
- [37] A sequential neural recommendation system exploiting BERT and LSTM on social media posts Scientific Figure on ResearchGate. Available from: https://www.researchgate.net/figure/BERT-model-architecture_fig2_372906672
- [38] Creo, Aldan, and Shushanta Pudasaini. "Evading AI-Generated Content Detectors using Homoglyphs." arXiv preprint arXiv:2406.11239 (2024).
- [39] Malik, Vijit, Ashwani Bhat, and Ashutosh Modi. "Adv-OLM: Generating textual adversaries via OLM." arXiv preprint arXiv:2101.08523 (2021).
- [40] Guo, Chuan, et al. "Gradient-based adversarial attacks against text transformers." arXiv preprint arXiv:2104.13733 (2021).

Generated using Undergraduate Thesis L^AT_EX Template, Version 1.4. Department of Computer Science and Engineering, Ahsanullah University of Science and Technology, Dhaka, Bangladesh.

This thesis was generated on Monday 23rd December, 2024 at 7:17pm.